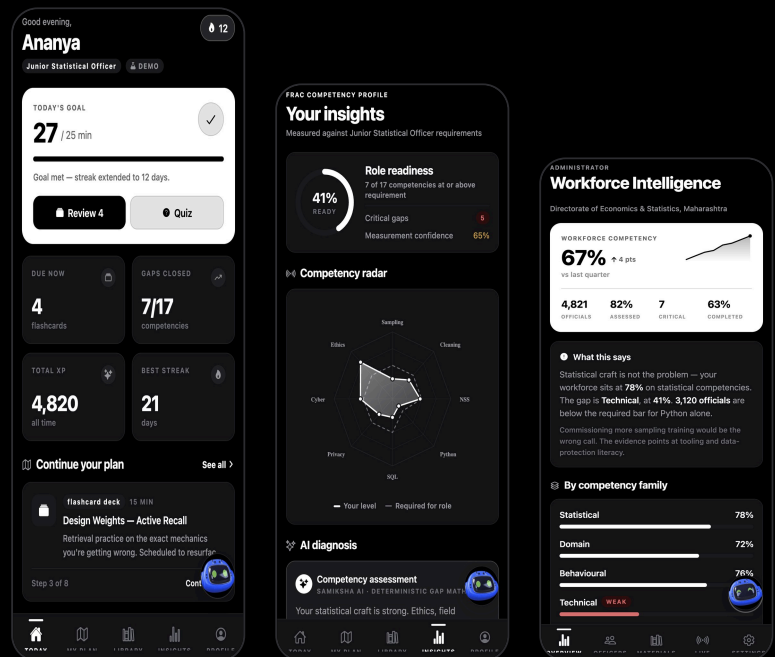


समीक्षा Samiksha



An AI-enabled capacity-building platform for India's Official Statistical System — it measures competency gaps, teaches what is missing, and gives every officer AI agents that know where they are weak.



Technical Documentation

Problem, solution, architecture, AI design, algorithms, security, operations

Deliverables

Officer & Administrator Android apps · Demo apps · Windows AI Workspace

Version

1.2 · September 2026
Ministry of Statistics & Programme Implementation (MoSPI)

What this document covers

This document is the complete technical and product record of Samiksha: the problem it answers, how it works for each user, how it is built, why each decision was taken, and what has been delivered.

1	Executive summary	3	12	Server functions reference	35
2	The problem	5	13	Code organisation	37
3	The solution	7	14	iGOT Karmayogi integration	39
4	How the app works — officer	9	15	Security, privacy & governance	40
5	How the app works — administrator	15	16	Deployment & operations	42
6	The AI Workspace & the competency loop	18	17	Quality & verification	45
7	Complete feature catalogue	20	18	What has been done, and why	47
8	System architecture	23	19	Limitations & roadmap	49
9	AI design & model choice	26	20	Anticipated questions	50
10	Algorithms & learning science	29	21	Glossary	52
11	Data model	32			

How to read this document

Sections 1–7 describe the product and are written for evaluators, administrators and policy readers. Sections 8–17 are the engineering reference: architecture, AI pipeline, algorithms, schema, code and operations. Sections 18–20 record what was built and why, the known limitations, and answers to the questions a review panel is most likely to ask. All screenshots are taken from the running application in its built-in demonstration mode, which uses a realistic seeded dataset for a Directorate of Economics & Statistics.

Executive summary

India's official statistics are produced by thousands of officers across MoSPI, the National Statistics Office, NSSTA and every state Directorate of Economics & Statistics. The skills that job needs are changing faster than traditional training can track. Samiksha tells each officer exactly where they fall short of their post, closes those gaps with evidence-based learning, and gives them AI agents for the repetitive analytical work — while learning from what they delegate.

39

FRAC COMPETENCIES, 4 FAMILIES

6

STATISTICAL POSTS, FI → DIRECTOR

42

DATABASE TABLES, 75 RLS POLICIES

10

SERVER-SIDE AI & ADMIN FUNCTIONS

5

INSTALLABLE APPS FROM ONE RELEASE

3

AI AGENTS PER OFFICER, COMPETENCY-AWARE

What Samiksha does

SIH26101 ASKS FOR	SAMIKSHA DELIVERS
Identify competency gaps	Every officer is measured against the FRAC competencies their post requires. Gap size and priority are calculated deterministically in the database — reproducible and auditable — and an adaptive test (Item Response Theory) measures ability in roughly half the questions a fixed test needs.
Recommend personalised training via iGOT Karmayogi	An adapter written against Karmayogi's published API contract matches courses to gaps and enforces course eligibility rules. The AI sequences a plan using a prerequisite graph and proven learning-science constraints, and states the reason for every step.
Generate quizzes and MCQs from uploaded material	PDF, PowerPoint, Word, video and YouTube/web links become quizzes, flashcards and timed mock tests. Every question must quote its source; the quote is verified against the document and resolved to a page or slide number.
Strengthen capacity building	Spaced repetition (FSRS-6), misconception tracking, confidence calibration, an AI tutor, an administrator operator assistant, workforce-level dashboards, and an AI Workspace whose usage feeds back into the competency record.

What makes it different

The maths is not the model's

Gap arithmetic is SQL. The AI explains the numbers in plain language; it never invents them. A training decision in government must survive the question "why was I assigned this?"

"No data" is not "weak"

Each competency carries a measurement confidence. When evidence is thin, Samiksha recommends *assessing*, not *training* — so budgets are not spent on courses for skills nobody has measured.

Every question is traceable

Generated questions carry a verbatim source quote that is mechanically checked and cited to a page. Each wrong option is labelled with the misconception it represents.

Learning and doing, joined

Agents are told where the officer is weak and explain the step that matters. Every delegation is recorded against a competency, turning "repeatedly hands Python work to an agent" into evidence of a gap that is costing real autonomy.

The deliverable, in one line

Two Android apps (Officer and Administrator) and two no-account demo apps built from one codebase, a Windows desktop AI Workspace, and a secured Supabase backend with ten server functions — all produced by an automated release pipeline, running on a single open-weight AI model chosen because it can be fine-tuned and hosted inside government infrastructure.

The problem

Problem statement SIH26101 — Ministry of Statistics & Programme Implementation

Develop an AI enabled learning platform that identifies competency gaps, recommends personalized training through integration with the iGOT Karmayogi ecosystem, and capable of generating Quizzes and Multiple choice questions (MCQs) from uploaded learning materials to strengthen capacity building in India's Official Statistical System.

Who is affected

The National Statistical System spans the Ministry of Statistics & Programme Implementation (MoSPI), the National Statistics Office (NSO), the National Statistical Systems Training Academy (NSSTA), and the Directorates of Economics & Statistics in every state. Officers range from Field Investigators conducting household enumeration to Directors who set divisional statistical policy. Their output — national accounts, price indices, PLFS, NSS surveys, SDG indicators — informs budgets, monetary policy and welfare programmes.

Why the current approach falls short

CHALLENGE	WHAT HAPPENS TODAY	CONSEQUENCE
Skill needs are shifting	Statistical craft (sampling, estimation, quality assurance) is still essential, but officers are now also expected to work in Python, R and SQL, use GIS, understand the DPDP Act and handle cybersecurity.	An excellent sampling statistician may be unable to query the database their data sits in.
Gaps are self-reported	Training needs are collected through forms and nominations. People systematically misjudge their own skill.	Training is assigned to the wrong people for the wrong topics.
Recommendation is generic	Course catalogues such as iGOT Karmayogi are large, but officers are not told which course, in which order, closes their specific gap.	Low completion; courses taken out of prerequisite order fail.
Assessment is expensive to author	Trainers write MCQs by hand from handbooks, slides and circulars.	Few assessments exist; material is read once and forgotten.
No workforce view	Administrators see enrolments and completions, not where the directorate is collectively weak.	Training budgets are committed without evidence of need or impact.
Learning is disconnected from work	AI tools, where used, simply do the work.	Productivity rises while capability does not; dependency is invisible.

Reading the statement carefully

The statement bundles **two AI engines** and **two audiences**, and it is easy to build only half:

Skill intelligence

Profile → prior → measurement → competency gaps → iGOT and NSSTA recommendations, sequenced by prerequisites.

Assessment engine

Upload PDF / PPTX / DOCX / video → grounded MCQ generation → page-cited, misconception-tagged questions that update the competency profile.

Learner

Wants to know: *what should I study next, and why?*

Administrator / nodal officer

Wants to know: *where is my workforce weak, how many people does it affect, what should we commission, and is it working?*

A system that answers only one engine or one audience is a quiz generator or a dashboard. Samiksha closes the loop: assessment changes the competency profile, which changes recommendations, which changes what is learned, which changes the next assessment — and the AI Workspace adds a second source of evidence from real work.

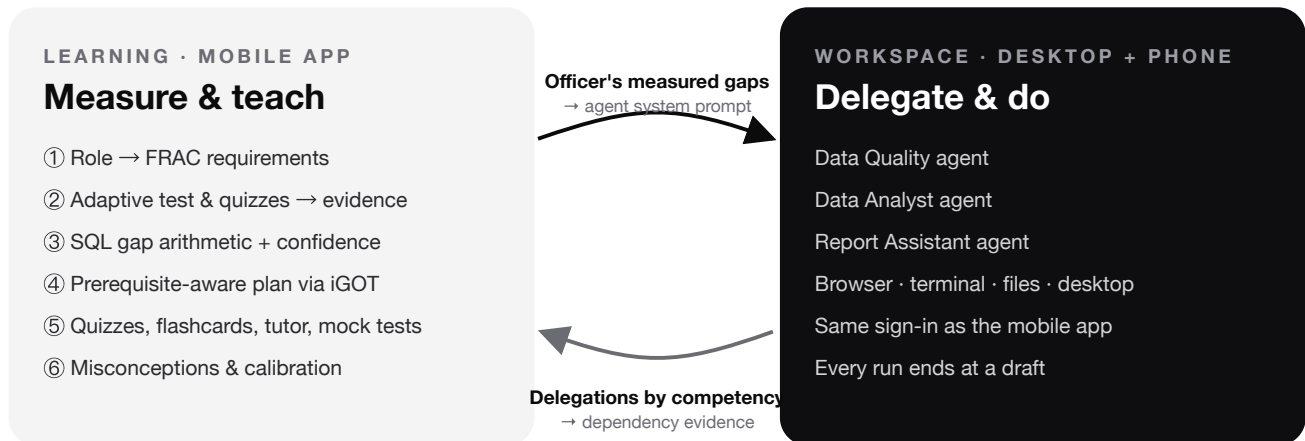
Design requirements we derived

1. **Defensible.** Any recommendation must be explainable from recorded evidence, not from a model's opinion.
2. **Honest about uncertainty.** Distinguish "not yet measured" from "measured and weak".
3. **Grounded.** Generated questions must be traceable to the source document.
4. **Private by construction.** Administrators see aggregates; officers' data is protected by database-level rules; no secret ever ships inside an app.
5. **Controlled access.** Government workforce data — no self-registration; accounts are provisioned against verified employee IDs.
6. **People decide.** Official statistics are published under a person's name, so AI drafts and proposes; humans confirm.
7. **Affordable at scale and sovereign.** Per-officer AI generation must cost fractions of a rupee, and the model must be one the government could fine-tune and host itself.
8. **Works in the field.** Android-first, resilient to poor connectivity, with an offline demonstration mode.

The solution

Samiksha is two halves that feed each other. **Learning** measures each officer against the competencies their post requires and teaches what is missing. **The Workspace** gives them AI agents for repetitive analytical work. **The loop** joins them — and the loop is the point.

THE COMPETENCY LOOP



"What you are handing to your agents" — shown to the officer, ranked where automation is covering a gap

Four products, one platform

PRODUCT	RUNS ON	WHO IT IS FOR, AND WHAT IT DOES
Samiksha (Officer)	Android	The learner app. Competency profile and radar, adaptive diagnostic, personalised plan with iGOT courses, department study material, AI practice (quiz, flashcards, mock test), spaced-repetition review, Samiksha AI tutor, and the mobile view of the AI Workspace.
Samiksha Admin	Android	Workforce intelligence for administrators and nodal officers: department-wide competency health, officer roster and records, live board, publishing study material, account provisioning, and Samiksha AI as an operator with confirm-before-act tools.
Samiksha Demo & Admin Demo	Android	The same two journeys over a seeded dataset with no account needed — so evaluators can install and explore immediately, even offline.
Samiksha Workspace	Windows desktop (also web)	AI agents with their own computers (browser, terminal, files, desktop) for dataset checks, indicator computation and report drafting. Signs in with the same credentials, knows the officer's gaps, and records every delegation.

Who does what

Officer

Signs in with issued credentials, confirms role and daily goal, takes the 5-minute adaptive check, follows the plan, practises on assigned material, asks Samiksha AI, and delegates work to agents.

Administrator

Provisions accounts against employee IDs, publishes material to departments or named officers, watches workforce competency health and live activity, and asks Samiksha AI for data or to prepare actions.

Samiksha AI

Explains, generates and sequences — always from recorded data and supplied material. It proposes changes; people confirm them. It never names its underlying model.

Guiding principles

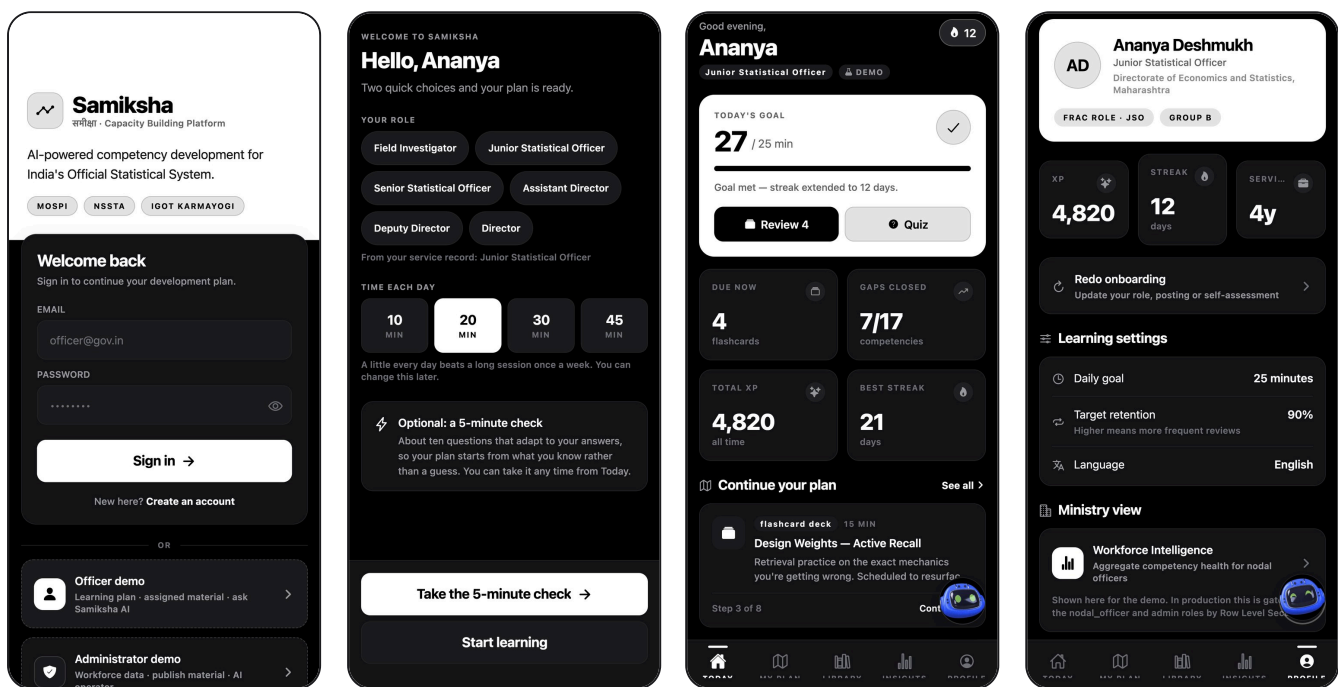
Evidence over opinion	Self-rating sets only a low-confidence prior. Measured answers raise confidence; the interface says so explicitly.
Deterministic core, AI at the edges	Scores, gaps, readiness, scheduling and adaptive item selection are formulas. The model explains, writes and sequences.
Agents draft, people decide	No agent instruction ends in sending, publishing or silently correcting a figure. Administrator actions via AI require explicit confirmation.
Monochrome, calm interface	White, black and greys; green and red appear only where colour carries meaning (answer correctness, critical gaps). Physics-based motion; a floating mascot provides context-aware help.
One source of identity	Supabase is the single source of truth for who an officer is and what they can do, across phone, desktop and web.

How the app works — the officer

The journey of Ananya Deshmukh, a Junior Statistical Officer at the Directorate of Economics & Statistics, Maharashtra, with four years of service. Every screen below is from the running app.

4.1 Sign in and a one-screen onboarding

Officers never self-register. An administrator creates the account against a verified employee ID, with department and years of service already filled in. On first sign-in the officer confirms two things — the post they are measured against (pre-selected from the provisioned designation) and how many minutes a day they can give — and may take an optional five-minute adaptive check. Self-rating was deliberately removed: it is the least reliable evidence available.



Sign in

Issued credentials, plus officer and administrator demo entries.

Onboarding

Role and daily goal — two taps.

Today

Goal, streak, due cards, priority gap, next plan step.

Profile

FRAC role, grade, learning settings, target retention.

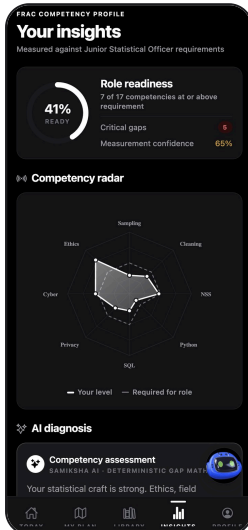
4.2 Today — the daily home

The Today screen answers "what should I do now?": progress against the daily goal and streak, flashcards due for review, gaps closed, total XP, the next step of the plan, the single highest-priority gap with its AI diagnosis, accuracy trend, and quick actions (upload material, ask a question, adaptive test, iGOT courses). The floating mascot offers a hint relevant to the current screen, never a generic greeting.

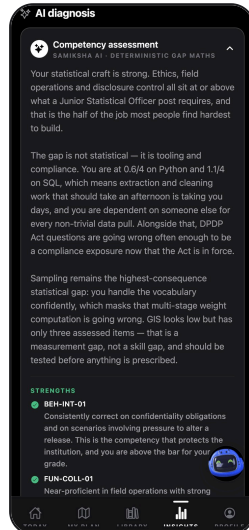
4.3 Insights — the competency profile

Insights shows the officer measured against their post's FRAC requirements: **role readiness** (competencies at or above requirement), critical gaps, and **measurement confidence**. A radar compares current level with required level across all four competency families. Below it:

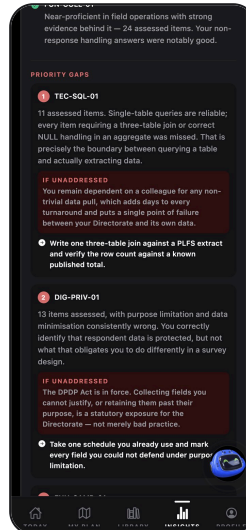
- **AI diagnosis** — a plain-language reading of the deterministic gap numbers: strengths, priority gaps with impact on the role, what happens if unaddressed, and a concrete first step.
- **What you believe instead** — recurring misconceptions, derived from which wrong option was chosen, with how many times and how recently.
- **What you are handing to your agents** — competencies the officer delegates to AI agents while still below the required level, with delegation count and time saved.
- **What you can learn next** — the learnable frontier: gaps whose prerequisites are already met, with a readiness score, and what each blocked competency needs first.
- **All competencies** — filterable by family, each with level bar, required marker, item count and confidence.



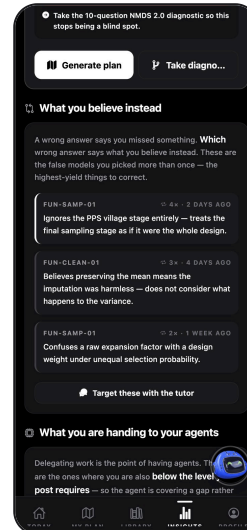
Readiness & radar
41% ready; 5 critical gaps;
65% confidence.



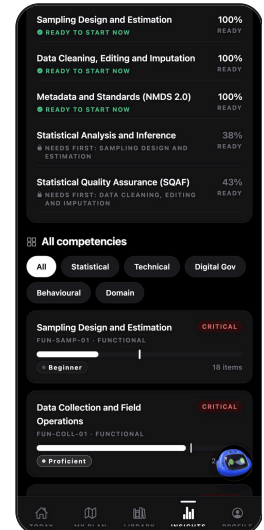
AI diagnosis
Explains the SQL-computed gaps.



Priority gaps
Rationale, impact, first step.



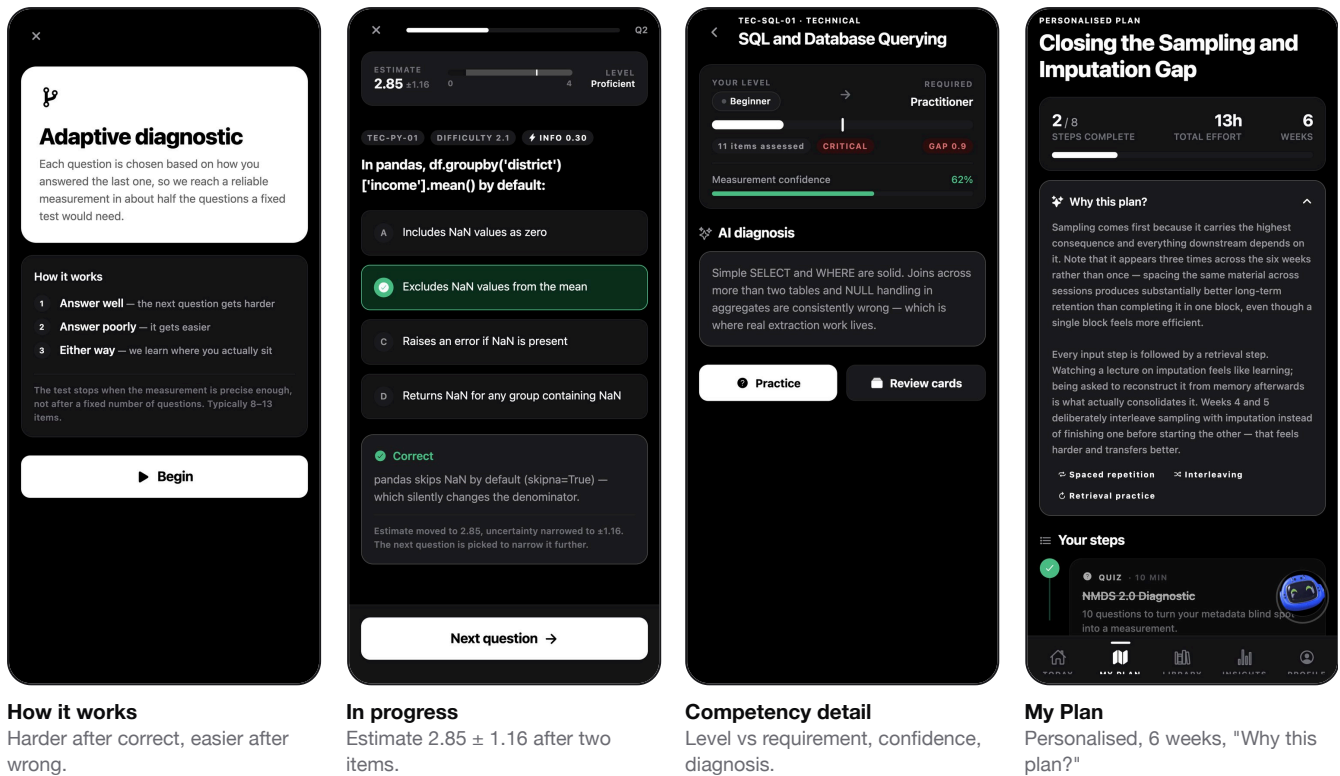
Misconceptions
The false models held.



Learn next
Prerequisite-aware readiness.

4.4 The adaptive diagnostic

Instead of a fixed 20-question test, each question is chosen to be maximally informative about where the officer sits, using a two-parameter Item Response Theory model. The screen shows the live ability estimate (0–4 scale), its uncertainty band narrowing with each answer, the implied level, and the information value of the current item. The test stops when the measurement is precise enough — typically 8–13 items. Results write directly into the competency scores with increased confidence.



4.5 My Plan — personalised training

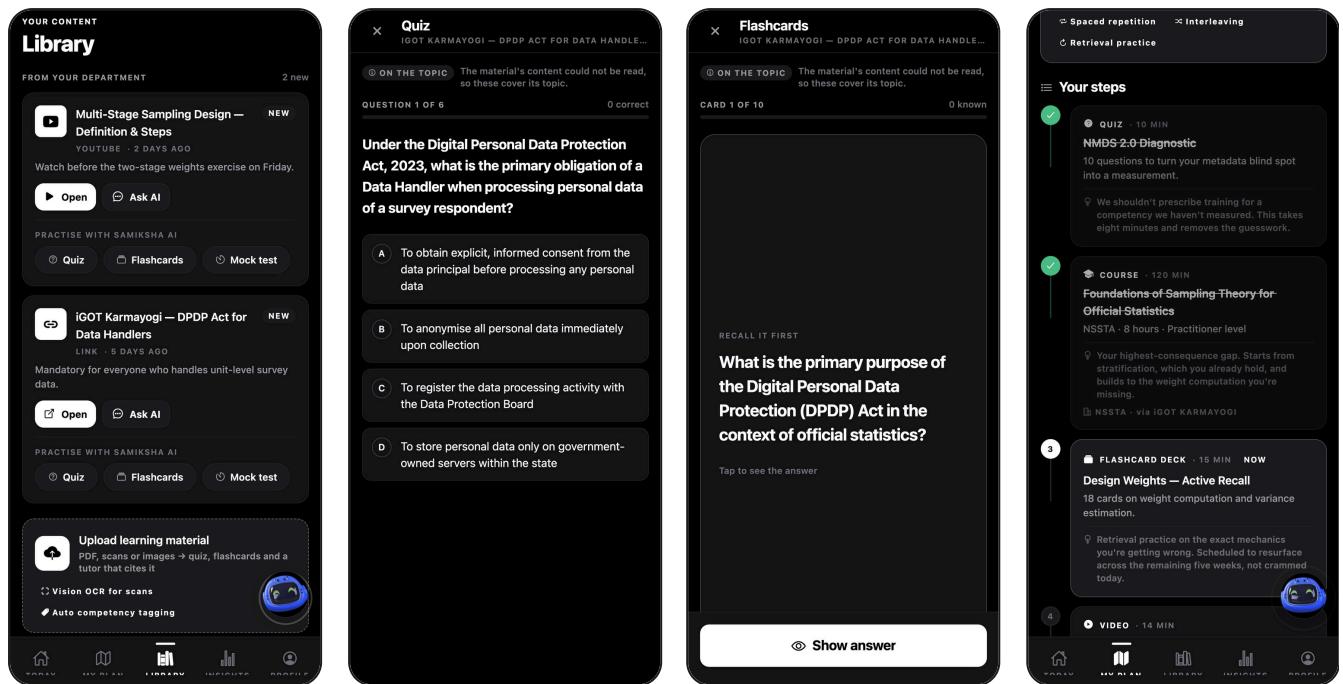
The plan is generated from the officer's gaps, the prerequisite graph and matching iGOT Karmayogi / NSSTA courses. It shows steps complete, total effort and weeks, and an expandable **Why this plan?** explaining the sequencing: spacing a topic across weeks, following every input step with a retrieval step, interleaving related competencies, and never scheduling a competency before its prerequisites. Each step carries its own *why this* tied to the officer's actual gap. Step types: iGOT course, curated video, quiz, flashcard deck, practice exercise and reflection.

4.6 Library — department material and AI practice

Material that administrators publish appears under **From your department**: PDFs, slide decks, Word documents, videos, YouTube and web links. Each card offers *Open*, *Ask AI* (Samiksha AI answers with that material in context), and three practice modes written by Samiksha AI from the material itself:

MODE	BEHAVIOUR	PURPOSE
Quiz	Six questions, one at a time, immediate feedback and explanation.	Learning through retrieval
Flashcards	Ten cards; recall first, then check; missed cards come round again.	Memorisation of key facts
Mock test	Fifteen timed questions, no feedback until submission; written as three parallel batches covering concepts, procedures and judgement.	Knowing where you stand before the departmental exam

The screen states what the questions rest on — the document itself, a video's description, a web page, or only the title ("On the topic") — because each deserves a different level of trust.



Library
Material from the department, with practice modes.

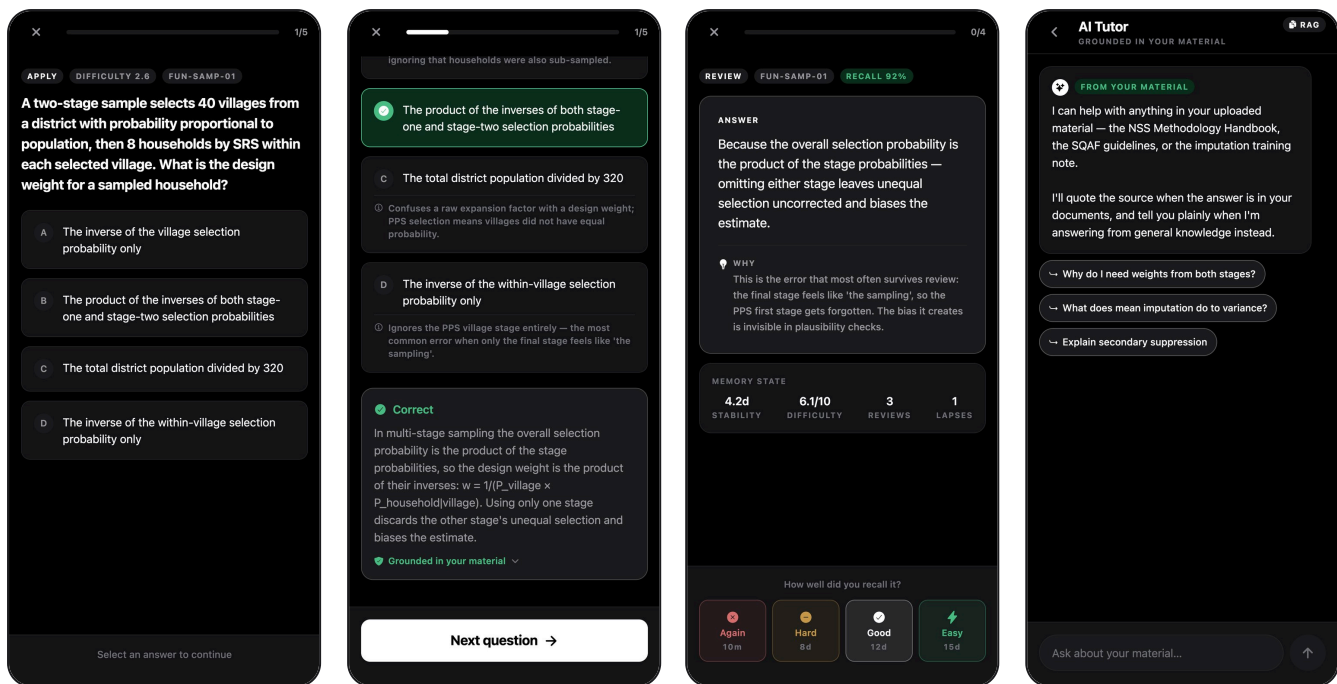
AI practice quiz
Generated live from assigned material.

AI flashcards
Recall first, then reveal.

Plan steps
Course, quiz, deck — each with a reason.

4.7 Grounded quizzes with confidence calibration

Competency quizzes show the Bloom level, calibrated difficulty and competency code of each question. After choosing an answer, the officer rates how sure they are (1 guessing – 5 certain) *before* the answer is revealed. Feedback then shows the correct answer, the explanation, why each distractor is wrong (the misconception it represents), and a **Grounded in your material** citation. The quiz summary reports calibration error and names overconfidence explicitly — a proven metacognitive intervention.



Question

Apply-level, difficulty 2.6, FUN-SAMP-01.

Feedback

Explanation, distractor rationale, source.

Spaced review

FSRS memory state and next intervals.

AI Tutor

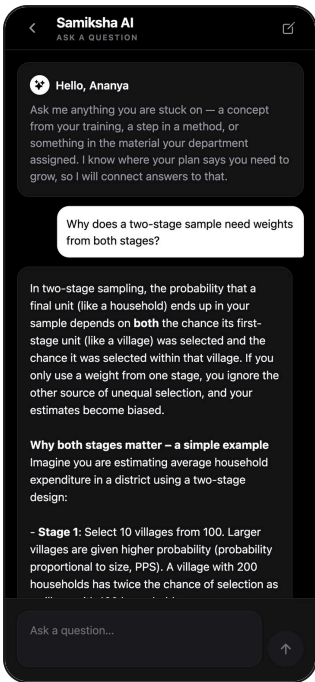
Retrieval-grounded; says when it is general knowledge.

4.8 Spaced-repetition review

Flashcards are scheduled by FSRS-6, a modern memory model. Each card shows its current recall probability; after "Show answer" the officer sees the answer, an elaboration explaining *why*, and the card's memory state (stability, difficulty, reviews, lapses). The four grade buttons display the actual next interval each would produce (e.g. Again 10m, Hard 8d, Good 12d, Easy 15d). Due cards are interleaved across competencies rather than drained one topic at a time.

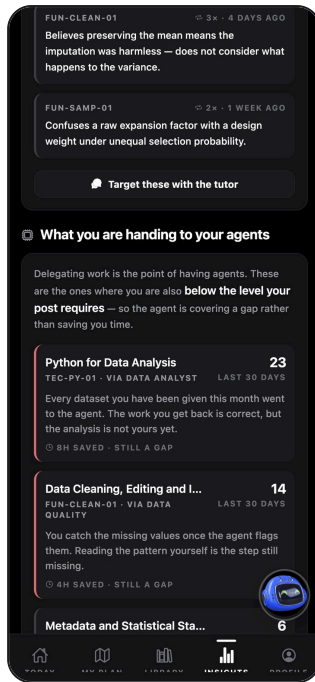
4.9 Samiksha AI — ask a question

Tapping the mascot opens Samiksha AI. For an officer it is a tutor that knows their gaps, their plan and the material their department assigned — so "explain this" is answered against their situation, with suggested questions and a "check your understanding" follow-up. The dedicated **AI Tutor** works over the officer's own uploads using retrieval: it cites the relevant passages and marks whether an answer comes from the material or from general knowledge.



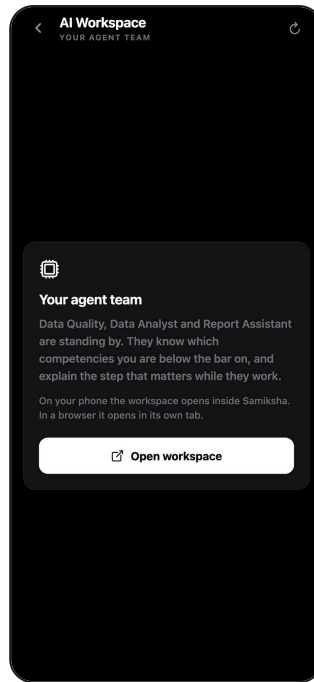
Samiksha AI

Live answer tied to assigned material.



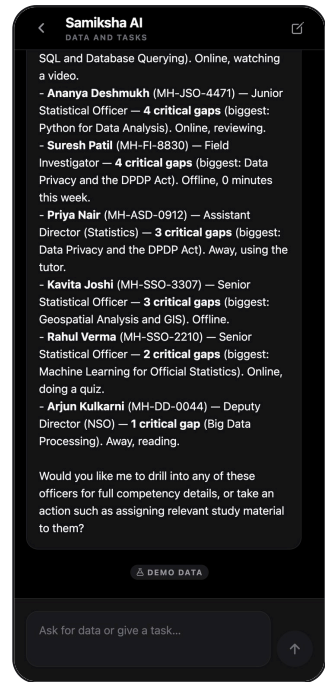
Agent dependency

23 Python tasks delegated while below the bar.



AI Workspace

Agent team, opened inside the app.



Operator mode

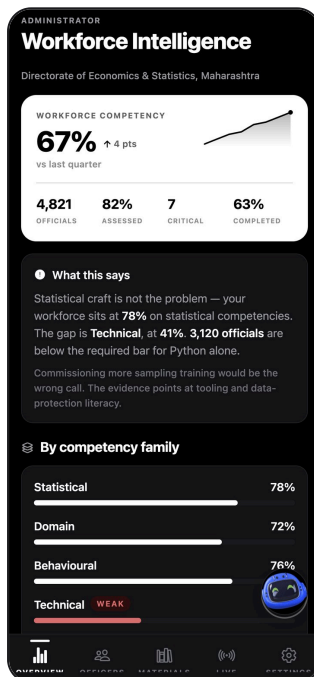
Administrator asks for live data.

How the app works — the administrator

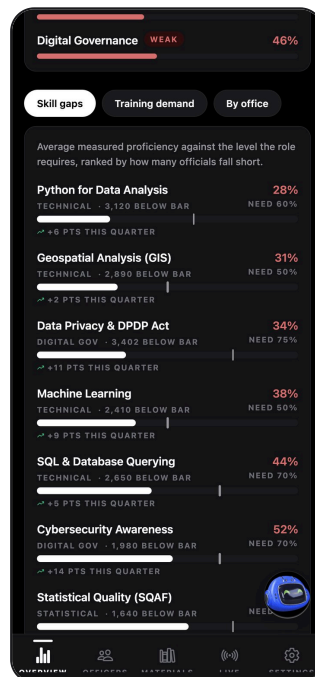
The administrator app is a separate application, not a tab bolted onto the learner app. An officer asks "what should I study next?"; an administrator asks "where are four thousand officers collectively weak, and is the training we commissioned working?" The role on the server decides which app opens.

5.1 Workforce Intelligence dashboard

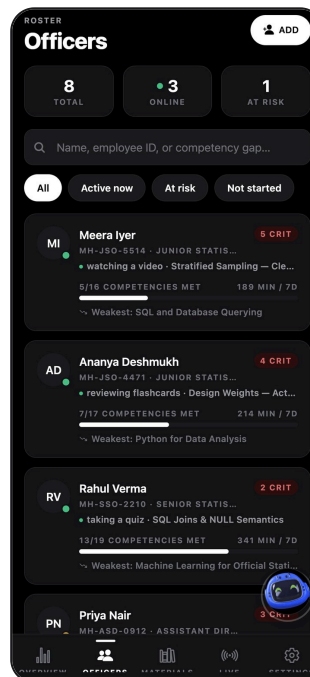
Headline workforce competency with quarter-on-quarter change, officials, share assessed, critical gaps and completion. **What this says** interprets the pattern — for example, that statistical craft is strong (78%) while technical competencies lag (41%), so commissioning more sampling training would be the wrong call. Competency families are broken down, then three views: **Skill gaps** (each competency's average proficiency against requirement, how many officials fall below the bar, and the quarterly trend), **Training demand** and **By office**. Every figure is an aggregate: the underlying database view groups before returning rows, so no individual is identifiable here.



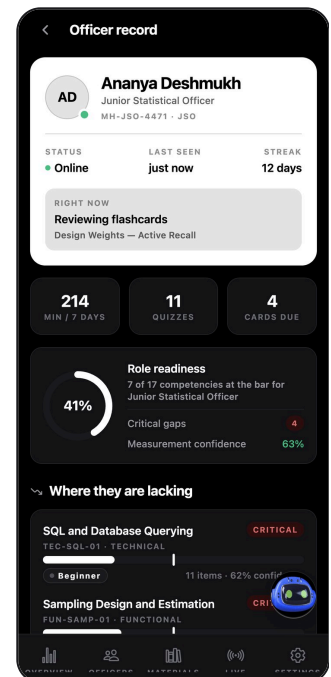
Workforce Intelligence
Aggregate competency health.



Skill gaps
Officials below bar, trend per competency.



Officer roster
Presence, gaps, weakest competency.



Officer record
Status, activity, readiness, gaps.

5.2 Officers and the officer record

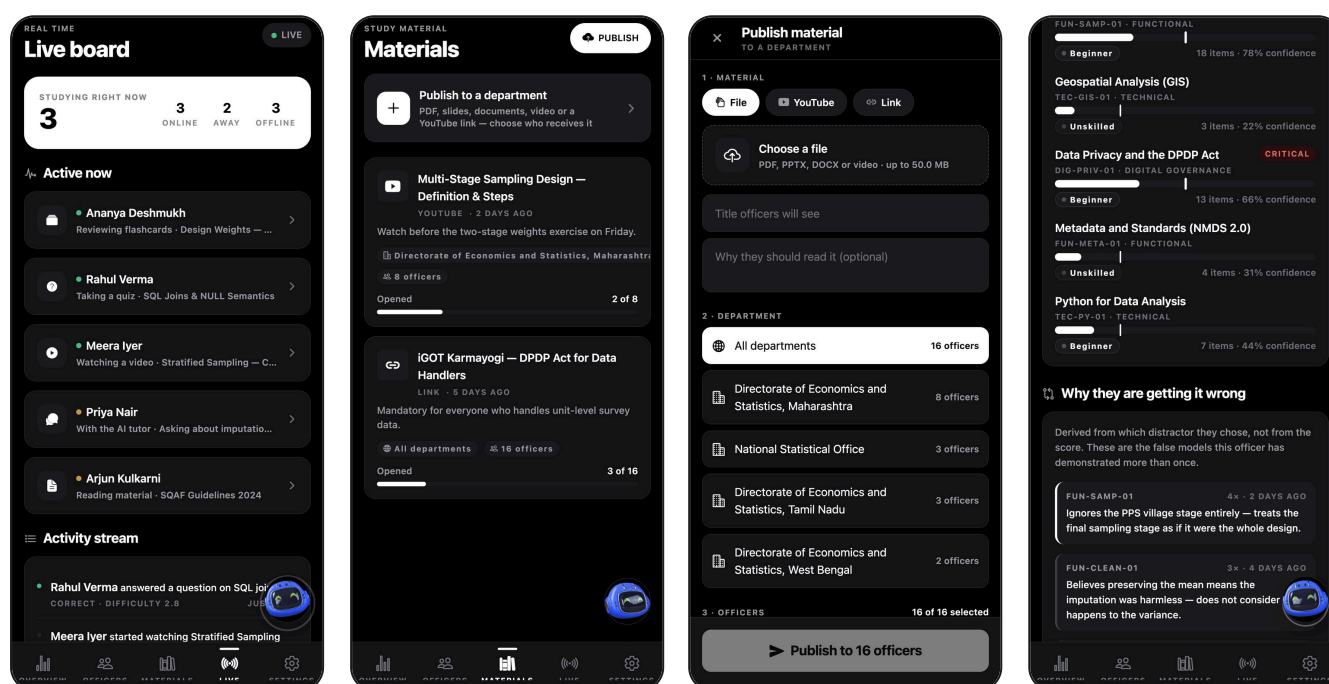
The roster is sorted by presence and filterable (All, Active now, At risk, Not started) with search by name, employee ID or competency gap. Each row shows competencies met, critical gaps, minutes this week, current activity and weakest competency. **Add** provisions a new officer. The officer record answers four questions on one screen: *Are they using it?* (status, last seen, streak, minutes, quizzes, cards due), *Where are they weak?* (ranked gaps against the FRAC role with confidence), *Why?* (the misconceptions they keep repeating), and *What happens next?* (the prescribed step and whether it is reachable). Account actions — reset password, deactivate, reactivate — are available here.

5.3 Live board

Who is using the platform right now and what they are doing — reviewing flashcards, taking a quiz, watching a video, asking the tutor — with an activity stream of answers and events. Presence is derived from a heartbeat timestamp stored in the database with realtime push on top: two minutes silent is *away*, ten is *offline*. A phone that loses signal cannot remain "online" forever and corrupt engagement figures.

5.4 Publishing study material

An administrator picks a file (PDF, PPTX, DOCX or video up to 50 MB, stored in a private bucket) or a YouTube / web link, gives it a title and reason, chooses a department or all departments, and ticks the officers who should receive it. Everyone starts ticked; exceptions — officers on deputation, someone who already completed the course — are unticked. Assignment is stored per officer, so the Materials tab shows who received each item and how many have actually opened it.



Live board

3 studying now; realtime activity.

Materials

Recipients and open rate.

Publish

File or link → department → officers.

Why they get it wrong

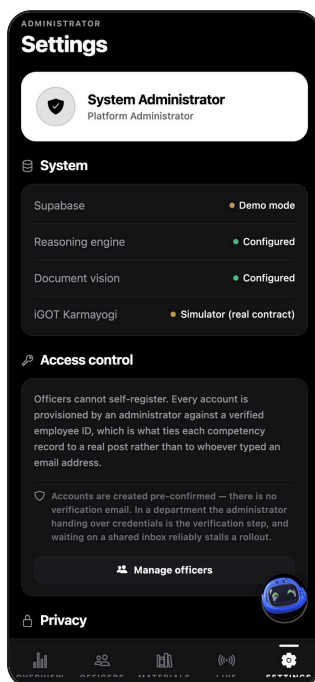
Misconceptions from distractor choices.

5.5 Samiksha AI as an operator

For administrators, Samiksha AI reads the platform through tools and can prepare actions. Read tools: platform overview, departments, officers (filter by department, search, critical gaps only), one officer in detail, competency gap summary, published materials. Action tools: publish a link, assign material, create an officer account, deactivate or reactivate an account, reset a password. **Every action returns a proposed-action card describing exactly what will happen; nothing runs until the administrator confirms.** Demo sessions send their own dataset, never touch the database, and are rate-limited.

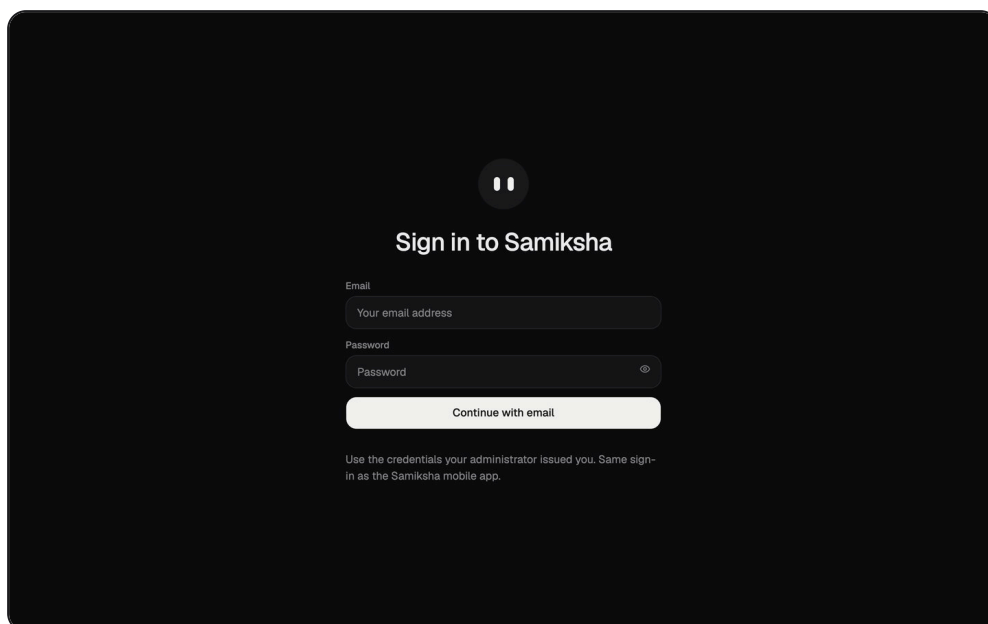
5.6 Settings and access control

System status (backend connection, reasoning engine, document vision, iGOT integration mode), access-control policy explained in-app, account management and privacy information. The engine's model name is never shown — officers see "Samiksha AI".



Settings

System status and access policy.



Samiksha Workspace (desktop)

Same credentials the administrator issued for the mobile app; self-registration is closed.

The AI Workspace & the competency loop

The tutor answers "help me understand this". The Workspace answers "do this for me". Keeping them one tap apart is deliberate: the moment an officer hands work over is exactly when the platform learns which competency they are leaning on.

6.1 What the Workspace is

A multi-service application — web interface, API, background worker, a sandbox supervisor, and an Electron desktop shell for Windows — where each officer has persistent AI agents with their own conversations, memory, routines and history. Agents can be given a computer: an isolated Linux container with a browser, terminal, file system and graphical desktop, so they can open a dataset, run analysis and produce files. Agents can also delegate to peer agents or short-lived sub-agents. On the phone, the mascot opens the same workspace inside Samiksha for reading results, replying and starting tasks.

6.2 The starter agent team

Officers never configure an assistant — the account an administrator provisioned is the whole setup. At first sign-in the server provisions three agents covering the work a statistical officer repeats, each mapped to the competencies a delegation to it is evidence about:

AGENT	COMPETENCIES	INSTRUCTIONS (SUMMARY)
Data Quality	FUN-CLEAN-01 FUN-QUAL-01	Checks incoming datasets: missing values by column and whether they look MCAR/MAR/MNAR, duplicates, inconsistent formats and units, implausible values, identifiers that do not join. States the consequence for the estimate. Never silently repairs data — proposes corrections and leaves the decision to the officer.
Data Analyst	TEC-PY-01 FUN-ANAL-01 FUN-SAMP-01 TEC-SQL-01	Cleans CSV/Excel survey data, applies the design weights the officer specifies, computes indicators and standard summary tables with charts. If the sampling design is unclear it asks rather than assuming simple random sampling. Shows method beside result and flags figures that moved materially.
Report Assistant	FUN-META-01 FUN-DISS-01 BEH-COM-01	Drafts periodic reports and correspondence against the approved template. Every figure must be traceable to a source table. Never invents, interpolates or rounds a figure; leaves marked gaps. Produces drafts only and never sends anything onward.

6.3 Identity bridge — one account everywhere

The Workspace keeps its own user rows (sessions, spaces and agents are keyed to them) but never runs a second, divergent credential store. Every sign-in is first proved against Supabase; the local row is then reconciled to match. Consequences:

- An administrator's password reset or deactivation in the mobile app takes effect on the next Workspace sign-in.

- The Supabase user ID is reused as the Workspace user ID, giving one identifier across phone, desktop and competency tables.
- Sign-up routes redirect to sign-in (`SIGNUPS_ENABLED=false`); accounts are pre-confirmed because the administrator handing over credentials is the verification step.
- If Supabase is unreachable the error surfaces as an outage, never silently as "wrong password".

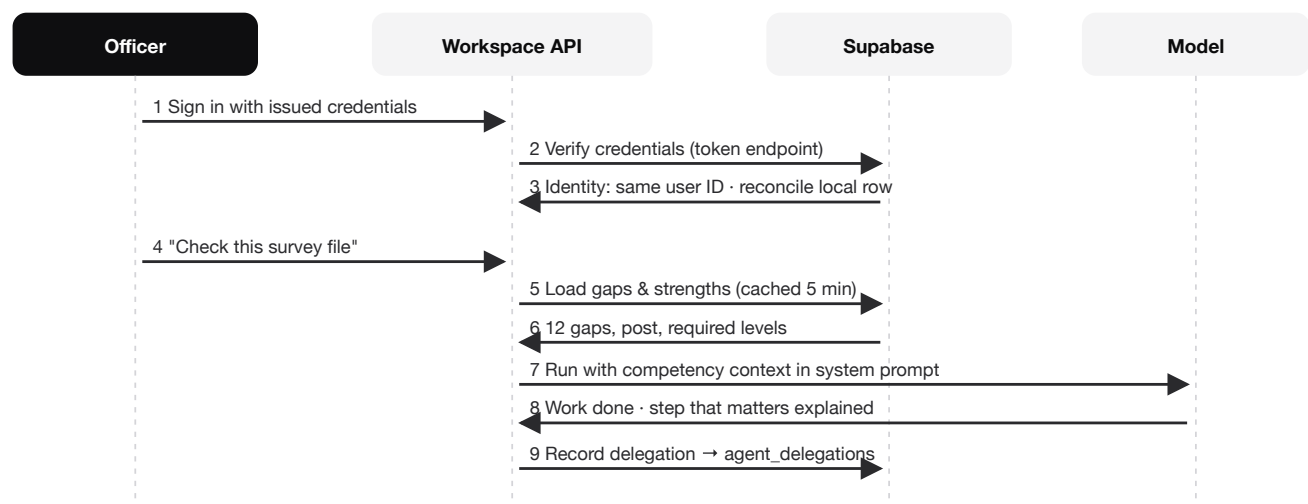


Figure — sign-in through the identity bridge, a competency-aware run, and delegation recorded as evidence.

6.4 Half one — agents are told where the officer stands

Before every run the Workspace reads the officer's post, their top gaps (current vs required level, family, criticality) and strengths from Supabase and adds them to the agent's system prompt. Where a task touches a competency the officer is below the bar on, the agent does the work *and shows the step that matters*; where the officer is already proficient, it just does the work. The lookup is best-effort and cached for five minutes: a reporting read must never be able to fail a run, and it should follow a finished diagnostic within minutes.

6.5 Half two — delegation becomes evidence

Completing a run writes an `agent_delegations` row: agent key and name, the primary competency the agent covers, task summary, duration and success. The view `v_officer_dependency` joins delegation volume (all-time and last 30 days, seconds automated) to the measured gap. Rows where the officer is already proficient score zero — delegating something you can do is good delegation, not a development need. The officer sees the result as **"What you are handing to your agents"**: e.g. *Python for Data Analysis — 23 tasks in 30 days via Data Analyst, 8 h saved, still a gap*. A gap measured in a diagnostic is hypothetical; twenty-three delegated Python jobs a month is the same gap costing real autonomy.

Why this matters

Automating work without regard to capability would make the platform better at doing the officer's job and worse at its purpose — making the officer capable of it. The loop keeps productivity and capacity building pointed in the same direction.

Complete feature catalogue

Competency intelligence

FEATURE	DETAIL
FRAC competency framework	39 competencies in four families — statistical (functional + domain), technical, digital governance, behavioural — mapped to six posts (Field Investigator, JSO, SSO, Assistant Director, Deputy Director, Director) with required proficiency, weight and criticality rising with seniority.
Five-level proficiency ladder	Unskilled · Beginner · Practitioner · Proficient · Expert (0–4), used consistently by scores, requirements, IRT ability and course difficulty.
Evidence-weighted scoring	Each answered item moves the score with a decaying learning rate; confidence grows with evidence as $n/(n+8)$.
Deterministic gap analysis	<code>recompute_competency_gaps()</code> — gap size and priority from requirement, weight, criticality and confidence.
Prerequisite graph (ZPD)	Competencies form a DAG with gate strength and minimum level; <code>get_zpd_competencies()</code> returns the learnable frontier ranked by priority × readiness.
Adaptive diagnostic	2PL IRT, maximum-information item selection, Newton–Raphson ability update, stop on precision; identical maths in the app and in SQL.
Misconception tracking	Distractors are labelled with the misconception they encode; choices are recorded and recurring beliefs surfaced to officer, tutor and administrator.
AI diagnosis	Plain-language interpretation of computed gaps: summary, strengths, priority gaps with role impact, first steps, recommended focus weeks.

Learning

FEATURE	DETAIL
Personalised learning path	AI-sequenced plan over iGOT / NSSTA courses, curated videos, quizzes, flashcard decks, practice and reflection, within the officer's daily-minute budget; every step has a stated reason.
Material ingestion	PDF text layer, PPTX slides and speaker notes, DOCX paragraphs, plain text; page-aware chunking (~1,200 chars, 150 overlap) with embeddings for retrieval; optional vision OCR for scanned pages.
Grounded MCQ generation	Strict JSON schema, verbatim source-quote verification with fuzzy fallback, structural validation, page/slide citation, Bloom and difficulty spread, question kinds: single, multi, true/false, assertion–reason.
Practice from any material	Quiz (6), flashcards (10), mock test (15, three hedged parallel batches), with extraction cached on the material and grounding disclosed.

FEATURE	DETAIL
Flashcards & FSRS-6	AI-written atomic cards with elaborations; faithful FSRS-6 scheduler with target retention setting; interleaved due queue.
Confidence calibration	Confidence captured before reveal; calibration error and overconfidence reported per quiz; live per-question difficulty calibration from real responses.
AI Tutor (RAG)	Retrieval over the officer's uploads, required citations, and a confidence field separating material-grounded from general-knowledge answers.
Curated videos	Official YouTube Data API search, AI assessment of pedagogical quality and topical fit, in-app playback through the official embedded player only.
Gamification that respects adults	Daily goal, streaks, XP and achievements (e.g. "Know What You Know" for calibration error under 0.15).

Administration

FEATURE	DETAIL
Account provisioning	Create against employee ID with department, designation, years of service; one-time readable temporary password; reset, deactivate, reactivate. Admin role verified server-side from the database before any operation.
Workforce Intelligence	Aggregate competency health by family, competency, office; officials below bar; quarterly trend; training demand.
Roster & officer record	Presence, engagement, ranked gaps, misconceptions, prescribed next step.
Live board	Heartbeat presence with realtime push and an activity stream.
Department study material	Publish files or links to all departments or one, per-officer recipients, open/completion tracking.
Samiksha AI operator	Read tools over the platform and confirm-before-act action tools.

AI Workspace

FEATURE	DETAIL
Persistent agents	Conversations, memory, routines and history; three competency-mapped starter agents provisioned at first sign-in.
Agent computers	Isolated containers with browser, terminal, files and graphical desktop; team and private computers; configurable CPU, memory and process limits.
Competency-aware runs	Officer gaps injected into every run; delegations recorded against competencies.
Voice	Dictation through the browser's built-in speech recognition, at no cost and with no key.
Desktop & mobile access	Windows installer; opened inside the phone app from the mascot.

Platform & experience

FEATURE	DETAIL
Floating mascot	Animated guide on every screen with context-specific prompts; opens Samiksha AI and the Workspace.
Motion system	Physics-based animation on the UI thread; haptics on device.
Light & dark themes	Monochrome design tokens following the system setting.
Demo mode	Full officer and administrator journeys over a seeded, subject-accurate dataset with no account; demo AI calls never touch the database.
Web build	The same app runs in a browser with web-safe dialogs and embeds.
Four APK variants + desktop installer	Built from one codebase by an automated release pipeline, each with distinct name, icon and package ID.

System architecture

Three deployable parts in one repository. The mobile app holds no secrets; Supabase is the single source of truth for identity and competency; every AI call happens server-side.

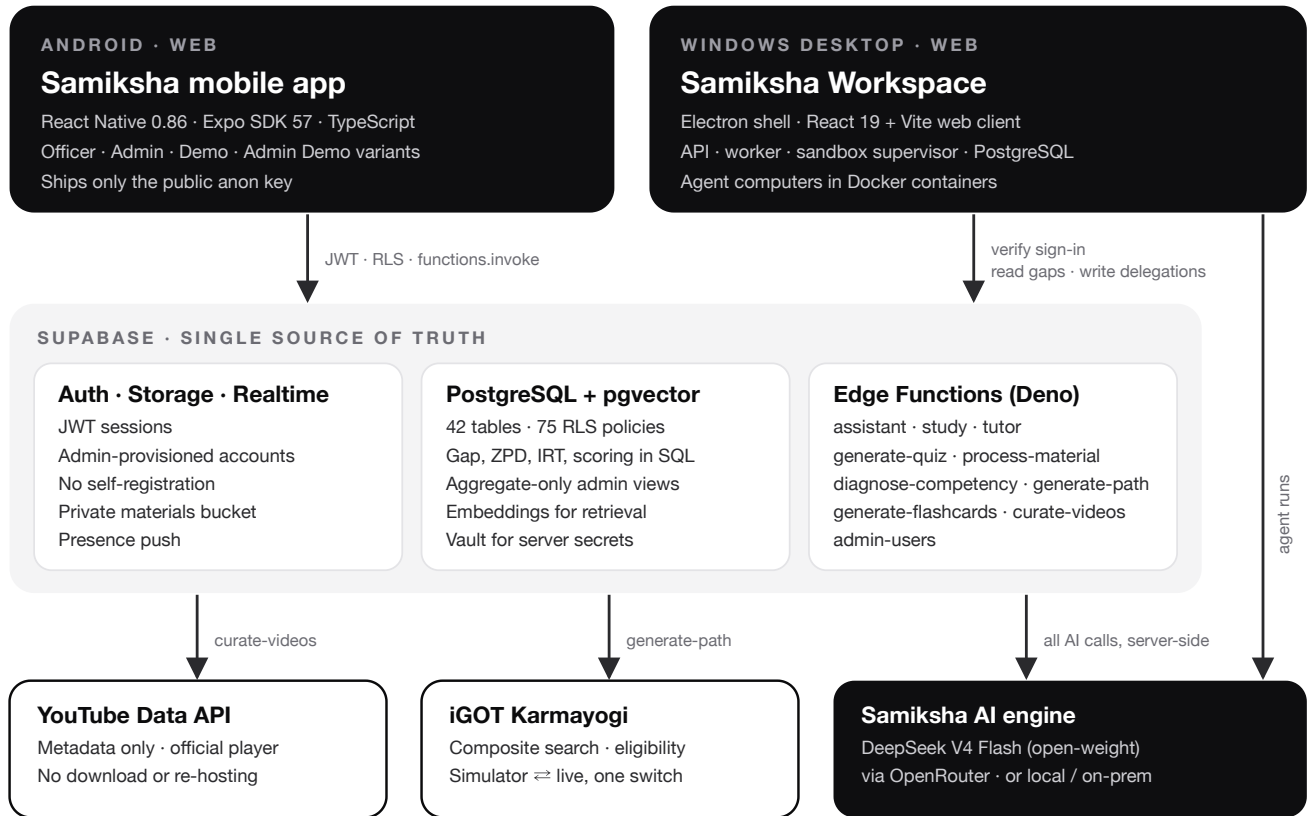


Figure — deployment architecture. Arrows show the direction of requests. Secrets exist only in the server-side boxes.

8.1 Repository layout

PATH	WHAT IT IS	RUNS ON
mobile/	Expo / React Native app; four build variants from one codebase	Android phone; browser (web build)
supabase/	15 ordered SQL migrations, a bundled <code>schema.sql</code> , 10 Edge Functions and shared modules	Supabase cloud
workspace/	pnpm monorepo: <code>apps/api</code> , <code>apps/web</code> , <code>apps/worker</code> , <code>apps/desktop</code> , <code>sandbox</code> infrastructure, shared packages	Windows PC (Docker Desktop, WSL2)
scripts/	<code>start.mjs</code> (one-command bring-up), <code>sync-env.mjs</code> (client-safe environment propagation)	Developer / demo machine

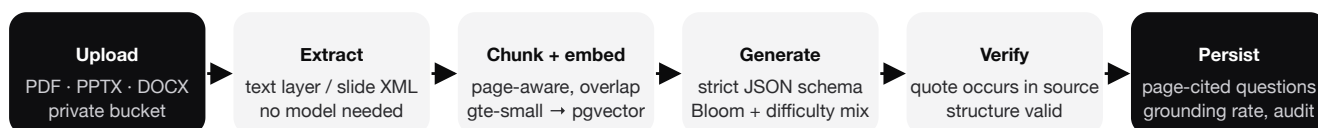
PATH	WHAT IT IS	RUNS ON
<code>.github/workflows/</code>	Release pipeline: four signed APKs and the Windows installer attached to a release	CI runners
<code>docs/</code>	This documentation, the presentation, setup runbooks, download page, screenshots	—

8.2 Technology stack

LAYER	TECHNOLOGIES	WHY
Mobile	React Native 0.86 (new architecture), Expo SDK 57, React 19.2, TypeScript, expo-router (typed routes), TanStack Query, Zustand, Reanimated 4, react-native-svg, expo-secure-store, expo-document-picker / image-picker, react-native-webview	One TypeScript codebase for Android and web; OTA-friendly; native performance for motion and charts.
Backend	Supabase: PostgreSQL 15+, pgvector, Row Level Security, Auth, Storage, Realtime, Vault, Deno Edge Functions	Security enforced in the database; serverless AI functions with secrets that never reach clients.
AI	DeepSeek V4 Flash through an OpenRouter gateway; built-in <code>gte-small</code> embeddings (384-d) in the edge runtime; local inference through Ollama	See Section 9.
Workspace	Node.js 22+/TypeScript, Hono API, React 19 + Vite + Tailwind web, Electron desktop, PostgreSQL + Prisma, Better Auth, background job worker, Docker sandbox supervisor, Turborepo, Vitest, Biome	Persistent agents with real computers, packaged as a desktop app.
Delivery	GitHub Actions (JDK 17, Android SDK, Gradle cache, Windows runner), Docker Compose	Reproducible, zero-cost builds and one-command demo bring-up.

8.3 Key request flows

Quiz generation from an uploaded document



Questions whose quote cannot be found are flagged and demoted; answering a quiz updates competency scores and records misconceptions.

Gap diagnosis — two deliberate layers

Deterministic (SQL)

Recompute gaps: required level minus measured score, $\times$ importance $\times$ criticality, damped by confidence. Reproducible, auditable, identical for identical records.

Interpretive (AI)

Given the computed numbers, profile and evidence, the model writes a summary, strengths, priority gaps with role impact and first steps — returned as strict JSON and cost-audited.

Personalised path

1. Recompute gaps in SQL and fetch readiness from the prerequisite graph.
2. Find courses for the gap competencies through the iGOT adapter (simulator or live), filtered by eligibility.
3. Ask the model to **sequence** — not list — under six learning-science constraints and strict rules (use only supplied course IDs and gap codes, stay within the minute budget, give a specific *why_this*).
4. Persist the path and items; progress triggers recalculate completion.

Practice from assigned material (**study**)

1. Resolve the material's words: read PDF/PPTX/DOCX directly; a web page's readable text; a YouTube video's title, channel and description (the phone may fetch and send it, since video platforms block cloud data-centre requests); a video file's title and description.
2. Cache extracted text on the material so the next officer does not pay for extraction again.
3. Generate under a strict schema; mock tests run as three 5-question batches in parallel, each **hedged** — if a call has not answered in 25 s a second is started and the first to finish wins — keeping well inside the platform's 150-second limit.
4. Return questions with the grounding type so the interface can disclose it.

Samiksha AI (**assistant**)

1. Mode by role: *learner* (tutor over gaps and assigned material) or *admin* (operator).
2. Tool-calling loop of up to six rounds; read tools execute immediately against data scoped to the administrator's organisation.
3. Write tools are intercepted: the function returns a *pending action* with a human-readable summary. Only a second request carrying **confirm** executes it.
4. Demo requests carry their own dataset, never read or write the database, and are rate-limited (120 calls per 10 minutes).

AI design & model choice

One pinned model powers every AI feature — the mobile functions and the Workspace agents: **DeepSeek V4 Flash**, accessed through an OpenRouter gateway. This is a design decision, not a shortcut.

9.1 Why DeepSeek V4 Flash

REASON	WHAT IT MEANS FOR SAMIKSHA
Open weights — easy to fine-tune	The model's weights are openly published under a permissive licence. That makes it straightforward to fine-tune with parameter-efficient methods (LoRA / QLoRA adapters) on MoSPI and NSSTA material — NSS methodology manuals, SQAF and NMDS guidance, national accounts handbooks, departmental exam papers and trainer-approved questions. A closed, API-only model cannot be adapted this way, and its behaviour can change without notice.
Sovereign hosting	Because the weights are available, the same model (base or fine-tuned) can be served inside government infrastructure such as NIC / MeghRaj data centres, so officer data and generated content need never leave Indian jurisdiction — important under the DPDP Act, 2023.
Efficient "Flash" architecture	A mixture-of-experts design activates only a fraction of its parameters per token. Inference is fast and cheap, and adapter fine-tuning needs far less GPU than dense frontier-scale models.
Very long context	A context window on the order of a million tokens means an entire 150-page handbook fits in one call. There is no chunk-stitching, so questions cannot contradict each other across chunk boundaries.
Cost at workforce scale	At roughly US\$0.09 per million input tokens, a 20-question quiz from a full PDF costs well under a cent. That is what makes per-officer, on-demand generation viable for tens of thousands of officers rather than a pilot for five.
Structured output & tools	Supports strict JSON-schema decoding (response shape guaranteed) and function calling (used by the administrator operator).
No lock-in	All calls go through one gateway module. The model is an environment variable (<code>MODEL_PRIMARY</code> , <code>PI_DEFAULT_MODEL</code>); the Workspace already runs against a small local model through Ollama (<code>PI_DEFAULT_PROVIDER=local</code>), proving the on-premises path.

9.2 Fine-tuning plan — the data is already being collected

Samiksha is built so that its own operation produces a fine-tuning dataset:

CAPTURED TODAY	BECOMES
<code>ai_generations</code> — every AI call with function, model, tokens, latency, cost and outcome	Prompt/response corpus and a cost/quality baseline to beat

CAPTURED TODAY	BECOMES
Questions with <code>source_quote</code> , grounding flag, Bloom level and observed difficulty from real responses	Supervised examples of well-grounded, correctly-calibrated MCQs; ungrounded ones as negatives
Distractor rationales and <code>misconception_events</code>	Training signal for writing distractors that diagnose real false beliefs in Indian statistical practice
Trainer / administrator review of material and questions	Gold-standard preference data for alignment (DPO)
<code>review_logs</code> (every flashcard review)	Per-population FSRS weight optimisation (not a model fine-tune, but the same principle)

The intended sequence: (1) continued pre-training or LoRA on the public statistical corpus of MoSPI/NSSTA; (2) supervised fine-tuning on trainer-approved questions, diagnoses and plans; (3) preference tuning from trainer edits; (4) evaluation against the grounding rate, structural validity and calibrated difficulty the pipeline already measures; (5) deployment on government infrastructure behind the same gateway, with no application changes.

9.3 How AI is used safely

Deterministic first

Parsing documents, computing gaps, readiness, ability and schedules are not AI problems and no model touches them. The model explains, writes and sequences.

Strict schemas

Every generation uses JSON Schema with `strict: true` , all properties required and no additional properties — an entire class of parsing bugs removed.

Mechanical grounding

Questions must quote the source verbatim; the quote is normalised (case, curly quotes, whitespace, punctuation) and searched in the document with a fuzzy fallback, then mapped to a page. Failures are flagged, and the grounding rate is reported.

Honest about sources

The tutor labels answers as from the material or general knowledge; practice discloses whether it rests on a document, a description, a web page or only a title.

Resilient calls

Jittered retry with backoff, request timeouts, hedged parallel batches for long generations, and graceful fallback so a slow lookup never fails a user action.

Audited & bounded

Every call is logged with token counts and cost; demo traffic is rate-limited; the model is told never to name itself, invent data or use formatting unsuited to a phone.

People decide

Administrator actions via AI are proposals requiring confirmation; agents stop at drafts and never publish or correct figures silently.

Text-only is explicit

A scanned PDF with no text layer is not silently indexed as empty. Pages with implausibly little text are escalated to vision OCR when a vision-capable model is configured, or rejected with an actionable message.

9.4 Prompt engineering highlights

- **Quiz generation** — self-contained stems (never "according to the passage"), a deliberate Apply / Analyse / Evaluate distribution, distractors that encode a named misconception, explanations of mechanism, calibrated difficulty on the 0–4 scale, and only competency codes from the supplied list.
- **Flashcards** — one idea per card, a cue that forces genuine retrieval rather than pattern-matching, and an elaboration explaining *why*.
- **Path generation** — "You are a learning scientist, not a content lister": spacing, interleaving, retrieval after every input, prerequisite order, start with a win, end with consolidation.
- **Video curation** — search results are ranked by the model for pedagogical quality and topical fit, because platform relevance optimises for watch time rather than correctness.
- **Samiksha AI persona** — plain text for a phone screen, precise with numbers, never invents data, never names the underlying model.

A provider-selection lesson

Before settling on the gateway, alternative development-phase providers were evaluated and dropped when catalogued models were not actually entitled for the account or timed out under load — discovered before any integration code was written. Separately, disabling reasoning on this model was found to route to a provider that took over five minutes; the configuration deliberately leaves it enabled.

Algorithms & learning science

10.1 Competency score update

Each answered item tagged to a competency implies an ability and moves the stored score with a learning rate that decays as evidence accumulates:

```
implied = correct ? min(4, difficulty + 0.6) : max(0, difficulty - 1.0)
 $\alpha = \max(0.08, 1 / (2 + n))$     - 0.5 at  $n = 0$ ,  $\approx 0.08$  at  $n = 20$ 
score' = clamp04( score · (1 -  $\alpha$ ) + implied ·  $\alpha$  )
confidence = (n + 1) / ((n + 1) + 8)    - saturates with evidence
```

The current proficiency level is derived from the score by trigger, so every screen agrees.

10.2 Gap and priority

```
gap_size = required_ordinal - current_score
priority = max(0, gap_size) × weight × (critical ? 1.5 : 1.0) × (0.5 + 0.5 × confidence)
```

The confidence damping is the key idea: an unmeasured competency with a large nominal gap ranks below a measured one, and the interface recommends *assessing* it first. Two officers with identical records always get identical results.

10.3 Zone of proximal development

Competencies form a directed acyclic graph (`competency_prerequisites`) with a gate *strength* (1.0 hard gate, 0.5 helpful) and minimum prerequisite level. Examples: Python requires SQL; Machine Learning requires Python and Statistical Analysis; Big Data requires SQL and Cloud; Open Data requires Dissemination and Data Privacy; Cloud requires Cybersecurity. **Readiness** is the strength-weighted fraction of prerequisites met, and the learnable frontier is ordered by:

```
rank = priority × (0.25 + 0.75 × readiness)
```

The planner is forbidden from scheduling a blocked competency before its blocker — teaching variance estimation to someone who cannot yet compute a design weight wastes time and confidence.

10.4 Adaptive testing — 2PL Item Response Theory

```
P(correct |  $\theta$ , b) = 1 / (1 + e-a( $\theta$  - b))    a = 1.2 (held constant)
I( $\theta$ ) = a2 · P · (1 - P)    - information peaks where difficulty  $\approx$  ability
 $\theta_0 = 2.0$ , SE0 = 1.6    - mid-scale prior, deliberately wide
 $\theta' = \theta + \Sigma a(u - P) / \Sigma I$     SE =  $\sqrt{1 / \Sigma I}$     - Newton–Raphson step
```

The next item is the unused item with maximum information at the current θ ; the test stops when SE is small enough or the item budget is reached. Ability lives on the same 0–4 scale as proficiency, so results drop straight into competency scores. The maths is implemented twice — in TypeScript for the app and in SQL (`item_information` , `next_adaptive_item` , `submit_adaptive_response`) — and both produce identical numbers. Discrimination is held constant because per-item estimation would overfit on early data; observed difficulty is recalibrated from real responses.

10.5 Spaced repetition — FSRS-6

The scheduler is a formula-for-formula implementation of FSRS-6 using the three-component memory model: **Stability** (days for recall to fall from 100% to 90%), **Difficulty** (1–10) and **Retrievability** (current recall probability). After each review (Again / Hard / Good / Easy) stability and difficulty are updated, and the next review is set for when retrievability decays to the officer's target retention (default 90%). FSRS typically needs 20–30% fewer reviews than the older SM-2 algorithm for the same retention. The implementation was verified against the reference implementation across test sequences with identical intervals:

REVIEW SEQUENCE	INTERVALS PRODUCED	MATCH
All Good	10m → 2d → 11d → 46d → 163d → 498d → 1348d → 3299d	Identical
All Easy	8d → 66d → 397d → 1875d → 7265d	Identical
Good x3 → Again → Good	10m → 2d → 11d → 10m → 2d → 5d	Identical

10.6 Calibration

```
calibration_error = mean over answers of | (confidence - 1)/4 - correct |
```

Overconfident answers (wrong with confidence ≥ 4) are counted and named. The "Know What You Know" achievement rewards calibration error below 0.15.

10.7 Misconception aggregation

Each distractor carries a rationale describing the false belief it encodes. `record_misconceptions()` writes an event for every wrong choice; `v_recurring_misconceptions` groups them per officer and competency. A belief shown once is noise; three times is the thing worth teaching against — surfaced to the officer, targeted by the tutor, and shown to administrators as "why they are getting it wrong".

10.8 Presence

Clients heartbeat into `user_presence` with their current activity. Status is derived, never trusted from the client: under 2 minutes online, under 10 away, otherwise offline.

10.9 The learning science applied

METHOD	WHAT THE EVIDENCE SAYS	WHERE IT LIVES IN SAMIKSHA
Spaced repetition	Distributed practice beats massed practice for long-term retention.	FSRS-6 scheduling; plans revisit a topic across weeks.
Retrieval practice	Recalling strengthens memory far more than re-reading.	Every input step in a plan is followed by a retrieval step; quizzes and flashcards from every material.
Interleaving	Mixing related topics improves transfer, though it feels harder.	Due cards rotate competencies round-robin; plans alternate related gaps.
Elaborative interrogation	Asking "why is this true" at encoding improves recall.	Every flashcard carries an elaboration; explanations state mechanism.
Metacognitive calibration	Knowing what you don't know predicts self-directed learning success.	Confidence before reveal; calibration error and overconfidence reported.
Desirable difficulties	Conditions that slow acquisition can improve retention.	Interleaving and expanding intervals applied deliberately and explained in-app.
Bloom's taxonomy	Recall-only assessment cannot distinguish competence levels.	Generated quizzes target Apply / Analyse / Evaluate; distribution shown.
Zone of proximal development	Learning is most effective just beyond current ability with prerequisites in place.	Prerequisite graph and readiness-weighted recommendations.
Testing effect & adaptive measurement	Well-targeted items measure precisely and teach while measuring.	2PL IRT adaptive diagnostic.

Data model

42 tables across 15 ordered migrations, every one deny-by-default under Row Level Security with 75 explicit policies. The complete schema is also bundled as a single paste-ready

`supabase/schema.sql`.

11.1 Tables by domain

DOMAIN	TABLES	PURPOSE
Identity & organisation	<code>organizations</code> <code>profiles</code> <code>user_presence</code> <code>notifications</code>	Ministries, NSO, NSSTA, state directorates (hierarchical); officer profile with role, employee code, designation, service years, goals, retention; live presence.
FRAC competency	<code>competencies</code> <code>job_roles</code> <code>activities</code> <code>role_competencies</code> <code>competency_prerequisites</code> <code>user_competency_scores</code> <code>competency_gaps</code>	Taxonomy, posts, required levels and weights, prerequisite DAG, evidence-weighted scores with confidence, computed gaps and priorities.
Content & material	<code>materials</code> <code>material_competencies</code> <code>material_chunks</code> <code>material_assignments</code> <code>courses</code> <code>course_competencies</code> <code>enrollments</code> <code>videos</code> <code>video_competencies</code> <code>video_progress</code>	Uploaded / published material with kind, audience and department; page-aware chunks with embeddings; per-officer assignment with opened/completed; iGOT catalogue and enrolment; curated videos.
Assessment	<code>quizzes</code> <code>questions</code> <code>quiz_attempts</code> <code>question_responses</code> <code>misconception_events</code> <code>adaptive_sessions</code> <code>adaptive_responses</code>	Grounded, Bloom-tagged questions with source quote and distractor rationales; attempts with calibration; responses with confidence; misconceptions; IRT sessions.
Learning engine	<code>decks</code> <code>flashcards</code> <code>review_logs</code> <code>study_sessions</code> <code>learning_paths</code> <code>path_items</code>	FSRS cards with stability/difficulty/state; full review history; sessions; AI paths with per-item reasons.
Engagement & analytics	<code>activity_events</code> <code>daily_stats</code> <code>achievements</code> <code>user_achievements</code>	Event stream, daily goal and streak rollups, achievements.
AI	<code>ai_generations</code> <code>tutor_conversations</code> <code>tutor_messages</code>	Audit of every AI call (tokens, cost, latency, status); tutor history with citations.
Competency loop	<code>agent_delegations</code>	Every Workspace run recorded against a competency.

11.2 Views

<code>v_org_competency_health</code>	Aggregate competency health per organisation — grouped before returning rows, so administrators cannot reach individual records through it.
<code>v_admin_officer_overview</code>	Roster row per officer: presence, competencies met, critical gaps, biggest gap, minutes in the last 7 days — for staff within the same organisation.
<code>v_recurring_misconceptions</code>	Misconceptions grouped by officer and competency with counts and recency.
<code>v_officer_dependency</code>	Delegation volume joined to measured gap; zero dependency where no gap exists.

11.3 Database functions

FUNCTION	ROLE
<code>recompute_competency_gaps(user)</code>	Deterministic gap and priority computation.
<code>apply_response_to_competency(...)</code>	Evidence-weighted score and confidence update (internal only; not callable by clients).
<code>submit_quiz_attempt(attempt)</code>	Grades an attempt, computes calibration, applies evidence, records misconceptions.
<code>get_zpd_competencies(user, limit)</code>	Learnable frontier with readiness and blockers.
<code>next_adaptive_item</code> · <code>submit_adaptive_response</code> · <code>complete_adaptive_session</code> · <code>item_information</code>	Server-side IRT adaptive testing.
<code>get_due_cards(limit, deck)</code>	Interleaved FSRS due queue.
<code>record_study_progress(...)</code> · <code>record_misconceptions(attempt)</code>	Daily stats, streaks, XP; misconception events.
<code>match_material_chunks(material, embedding, k, threshold)</code>	Vector similarity retrieval for the tutor.
<code>assign_material</code> · <code>can_read_material</code> · <code>can_read_quiz</code>	Per-officer assignment and access checks.
<code>admin_directory</code> · <code>heartbeat</code> · <code>presence_status</code>	Administrator directory and presence.
<code>is_staff</code> · <code>current_app_role</code> · <code>current_org_id</code> · <code>same_org_as</code>	Security helpers used inside RLS policies.
<code>get_service_secret(name)</code>	Reads server secrets from Vault; granted to the service role only.

FUNCTION	ROLE
Triggers: <code>handle_new_user</code> , <code>tg_sync_competency_level</code> , <code>tg_update_question_stats</code> , <code>tg_recalc_path_progress</code> , <code>tg_set_updated_at</code>	Profile creation on sign-up, level sync, live difficulty calibration, path progress, timestamps.

11.4 Migration history

0001	Extensions, enums (roles, competency families, proficiency ladder, Bloom levels, question kinds, FSRS states), organisations, profiles
0002	FRAC competency model and gap computation
0003	Content: materials, chunks with embeddings, courses, videos
0004	Assessment and quizzes with calibration and evidence updates
0005	Learning engine: decks, FSRS cards, review logs, paths
0006	Analytics, achievements, AI audit, tutor, aggregate views
0007	Row Level Security policies
0008	Seed: FRAC competencies, six posts, role requirements, achievements
0009	Prerequisite graph (ZPD) and misconception tracking
0010	Technical and digital-governance competencies and prerequisites
0011	Function grant hardening (revoke from anonymous and public)
0012	Presence, administrator provisioning, adaptive assessment
0013	Agent delegation loop
0014	Department study materials and per-officer assignment
0015	Restore EXECUTE on RLS helper functions for signed-in users (see §17)

Server functions reference

All functions are Deno Edge Functions under `POST /functions/v1/<name>`. They authenticate the caller from the JWT, use a user-scoped client (RLS applies) for user data, and reserve the service-role client for trusted writes such as audit logs.

FUNCTION	INPUT	BEHAVIOUR
assistant	<code>{ mode, messages, confirm?, demo? }</code>	Samiksha AI. Learner tutor over gaps and assigned material; administrator operator with 6 read and 5 confirm-before-act tools; up to 6 tool rounds; demo sessions rate-limited and database-free.
study	<code>{ mode: quiz flashcards mock, material_id? material?, source_text? }</code>	Practice from assigned material; extraction with caching; grounding type in response; hedged parallel batches for mock tests.
process-material	<code>{ material_id }</code>	Upload → extract (PDF, PPTX, DOCX, text; OCR fallback) → page-aware chunk → embed → AI analysis (summary, key concepts, competency tagging) → ready.
generate-quiz	<code>{ material_id raw_text competency_code, count, difficulty mix }</code>	Strict-schema MCQs, grounding and structural checks, page citation, Bloom/difficulty spread, grounding rate.
generate-flashcards	<code>{ material_id raw_text competency_code, count ≤ 40, deck }</code>	Atomic retrieval cards with elaborations into an FSRS deck.
diagnose-competency	—	Recomputes gaps in SQL, gathers evidence, returns AI interpretation (summary, strengths, priority gaps, first steps, focus weeks).
generate-path	<code>{ focus_competency_codes?, weeks = 6, max_items = 12 }</code>	Gaps + readiness + iGOT courses → learning-science sequencing → persisted path with reasons.
tutor	<code>{ conversation_id?, message, material_id?, competency_code? }</code>	Retrieval-augmented Socratic tutor with citations and a material-vs-general confidence label.
curate-videos	<code>{ competency / topic }</code>	Official YouTube Data API search (embeddable, strict safe search, 4–20 min) → AI quality and fit assessment → stored metadata for in-app playback.
admin-users	<code>{ action: create list reset_password deactivate reactivate, ... }</code>	Verifies admin role from the database before reading input; creates pre-confirmed accounts with unambiguous temporary passwords (no 0/O or 1/I/l).

Shared modules

<code>_shared/openrouter.ts</code>	Model gateway: chat with strict schemas, tool calling, retries with jittered backoff, timeouts, vision routing, generation logging with cost.
<code>_shared/schemas.ts</code>	JSON Schemas for quizzes, flashcards, diagnosis, learning paths, tutor replies, material analysis, video curation.
<code>_shared/igot.ts</code>	iGOT Karmayogi adapter (Section 14).
<code>_shared/officedocs.ts</code>	PPTX and DOCX text extraction from the ZIP/XML directly, slide order preserved, speaker notes included, entities decoded.
<code>_shared/supabase.ts</code> · <code>cors.ts</code>	Admin and user-scoped clients, <code>requireUser</code> , CORS and JSON responses.

Code organisation

13.1 Mobile app (`mobile/`)

Screens — `app/` (expo-router)

ROUTE	SCREEN
<code>index.tsx</code> · <code>_layout.tsx</code>	Entry router: demo build first-open, sign-in redirect, administrator vs officer application, onboarding gate; providers.
<code>(auth)/sign-in.tsx</code>	Credential sign-in; officer and administrator demo entries.
<code>onboarding.tsx</code>	Role (matched from designation) and daily goal; optional 5-minute check.
<code>(tabs)/index</code> · <code>path</code> · <code>library</code> · <code>insights</code> · <code>profile</code>	Officer tabs: Today, My Plan, Library, Insights, Profile.
<code>assessment.tsx</code>	Adaptive diagnostic with live estimate and uncertainty band.
<code>quiz/[id].tsx</code>	Quiz with confidence capture, feedback, citations, calibration summary.
<code>review.tsx</code>	FSRS flashcard review with memory state and interval preview.
<code>practice.tsx</code>	Quiz / flashcards / mock test from assigned material.
<code>competency/[code].tsx</code>	Competency detail: level, requirement, confidence, diagnosis, actions.
<code>assistant.tsx</code> · <code>tutor.tsx</code>	Samiksha AI (learner / operator with action cards) · RAG tutor.
<code>video/[id].tsx</code>	Embedded official player (iframe on web, WebView on device) with practice actions.
<code>workspace.tsx</code>	AI Workspace inside the app (launch card on web).
<code>publish-material.tsx</code>	Administrator publishing flow.
<code>(admin)/index</code> · <code>officers</code> · <code>officer/[id]</code> · <code>materials</code> · <code>live</code> · <code>settings</code>	Administrator application.

Library code — `src/`

<code>lib/supabase.ts</code>	Client with secure session storage; configuration detection.
<code>lib/api.ts</code>	Typed data access, function invocation, presence heartbeat, admin operations.
<code>lib/assistant.ts</code> · <code>lib/study.ts</code>	Samiksha AI and practice clients (including on-device YouTube description fetch).

<code>lib/materials.ts</code>	Material kinds, upload limits (50 MB), publishing, demo departments.
<code>lib/fsrs.ts</code>	FSRS-6 scheduler.
<code>lib/irt.ts</code>	2PL IRT: probability, information, ability update, item selection, stopping rule.
<code>lib/demo.ts</code>	Subject-accurate offline dataset: officer profile, 17 role competencies, plan, quiz, cards, adaptive pool, officers, workforce aggregates.
<code>lib/dialog.ts</code>	<code>confirmAsync</code> / <code>notify</code> — cross-platform dialogs (native alerts do nothing on web).
<code>lib/variant.ts</code>	Build variant (learner, admin, demo, demo-admin) — cosmetic and navigational only; never grants access.
<code>store/session.tsx</code>	Session, profile, demo mode, role routing, activity reporting.
<code>components/ui.tsx</code> · <code>charts.tsx</code> · <code>motion.tsx</code> · <code>FloatingMascot.tsx</code>	UI kit, SVG radar/rings/trends, physics-based animation kit, context-aware mascot.
<code>theme/tokens.ts</code>	Monochrome design tokens for light and dark.

13.2 Workspace (`workspace/`)

PART	RESPONSIBILITY
<code>apps/web</code>	React web client: agent conversations, computers, routines, settings, voice dictation.
<code>apps/api</code>	HTTP/RPC API, authentication, agent and space management.
<code>apps/worker</code>	Background jobs: agent runs, routines, notifications.
<code>apps/desktop</code>	Electron shell and Windows installer running the local stack.
<code>infra/sandboxes</code> · <code>infra/compose</code>	Agent computer image (X11 desktop, browser, terminal) and Docker Compose deployment; sandbox supervisor.
<code>packages/auth/src/supabase-identity.ts</code>	Identity bridge — credentials verified against Supabase, local row reconciled, shared user ID.
<code>packages/adapters/src/competency.ts</code>	Loads the officer's competency profile, renders it into the system prompt, records delegations.
<code>packages/adapters/src/executor.ts</code>	Run executor: loads competency context alongside memory and tools; records the delegation on completion.
<code>packages/db/src/default-agents.ts</code>	The three starter agents and their competency mappings; idempotent provisioning.
<code>packages/*</code>	Contracts, core domain, database (Prisma), logging, memory, UI tokens and components, chat UI, test kit.

iGOT Karmayogi integration

iGOT Karmayogi is a closed platform for civil servants; API credentials require an institutional arrangement. Samiksha's adapter is therefore written against Karmayogi Bharat's publicly documented API contracts, so switching from simulator to production is a configuration change, not a rewrite.

14.1 Endpoints targeted

ENDPOINT	USE
POST <code>/api/composite/v4/search</code>	Content search for courses, programmes and curated programmes matching gap competencies.
GET <code>/api/accessSettings/read/{id}</code>	Per-course access eligibility.
GET <code>/api/user/private/v1/read/{id}</code>	Learner profile (root organisation, profile verification status).
POST <code>/api/private/user/v1/search</code>	Ministry/department administrator lookup.

14.2 Two details that a naive integration gets wrong

No `primaryCategory` filter

Karmayogi's taxonomy includes course-like categories beyond "Course" and "Program" — notably "Curated Program". Filtering on primary category silently drops live courses, so the adapter does not.

`secureSettings` eligibility

Moderated courses restrict enrolment to learners whose root organisation / ministry appears in `secureSettings.organisation`, and — when `isVerifiedKarmayogi` is "Yes" — to verified profiles. The adapter enforces this, so recommendations never surface a course the officer cannot open.

14.3 Modes

IGOT_MODE=mock

A seeded catalogue of 23 courses built from published NSSTA / MoSPI curricula, tagged to competency codes with duration, level, rating and provider.

IGOT_MODE=live

The same code paths call the real endpoints with institutional credentials.

Normalisation

Both modes return the same `IgotCourse` shape (id, title, description, provider, duration, language, difficulty, rating, enrolled count, competency codes), so nothing downstream changes.

NSSTA TPAC programmes appear alongside iGOT courses. The learning-path generator may only reference course IDs the adapter returned, and each course step shows its provider and level.

Security, privacy & governance

15.1 Secret boundary

- **The app ships only the public URL and anon key.** An APK is trivially decompiled, so anything compiled into it is public by definition. Row Level Security, not obscurity, protects data.
- Only `EXP0_PUBLIC_*` values reach the app. `scripts/sync-env.mjs` refuses to run if a server secret (service role key, model key, YouTube key, iGOT key) ever carries that prefix — the boundary is enforced by a machine rather than memory.
- Server secrets live as Edge Function secrets or in Supabase Vault, read through `get_service_secret()` which is granted to the service role only. The model key can be rotated with a single SQL statement, no redeploy.
- The release pipeline verifies the APK knows its backend and stops rather than ship an app that cannot connect.

15.2 Database security

- All 42 tables are deny-by-default with 75 explicit policies: officers read and write their own rows; staff read within their own organisation; administrators see aggregates.
- `auth.uid()` is wrapped as `(select auth.uid())` so Postgres evaluates it once per statement instead of once per row.
- Functions use `security invoker` and an empty `search_path`; `execute` is revoked from `public` and `anon`, and internal functions are revoked from signed-in users too.
- Views that feed administrator screens use `security_invoker` and aggregate before returning rows.
- Presence status is derived server-side; client-supplied status is never trusted.
- Materials are stored in a private bucket; access is checked per officer assignment.

15.3 Access model

ROLE	CAN
Learner	Own profile, scores, gaps, attempts, cards, paths, tutor, delegations; material assigned to them.
Trainer	Author and review content and questions (NSSTA faculty).
Nodal officer	Aggregate dashboards for their organisation.
Administrator	Provision and manage accounts, publish and assign material, roster and officer records within scope, operator AI.

The build variant never grants access: an officer who installs the administrator app still receives the officer experience because the role comes from the server on every request.

15.4 Privacy and the DPDP Act, 2023

- **Purpose limitation** — data collected is what capacity building needs: role, evidence of competency, learning activity.
- **Data minimisation for oversight** — workforce views are aggregate by construction.
- **Controlled onboarding** — no self-registration; accounts tied to verified employee IDs.
- **Sovereignty path** — the open-weight model can be hosted on government infrastructure (Section 9).
- **Demo isolation** — demo AI requests carry their own data and never read or write the database.

15.5 AI governance

- **Agents draft, people decide** — no instruction ends in sending, publishing or silently correcting a figure.
- Administrator AI actions are proposals with a readable summary, executed only on confirmation.
- Every AI call is audited with tokens, cost and outcome.
- Grounding is verified mechanically and disclosed to users.
- Model names never appear in the interface; officers see "Samiksha AI".
- Video content is played through the official embedded player only; nothing is downloaded or re-hosted.

Deployment & operations

16.1 Install the apps

1. Download the APK from the repository's latest release: **Samiksha-Officer**, **Samiksha-Admin**, **Samiksha-Officer-Demo** or **Samiksha-Admin-Demo** (Android 7.0+).
2. Allow "Install unknown apps" for the browser or file manager, then install. All four can sit side by side — each has its own package ID.
3. Demo apps open straight into their journey; the others sign in with administrator-issued credentials.
4. For the desktop agent workspace, run **Samiksha-Workspace-Setup.exe** on Windows.

16.2 Bring the whole stack up — one command

```
node scripts/start.mjs          # containers (default)
node scripts/start.mjs --native # services on the host; agent computers still use Docker
```

The script finds the machine's address on the local network (skipping Docker, WSL and virtual adapters), writes it into every file that must agree about it, generates missing secrets, checks the model server when running locally, and starts the services. It is safe to re-run; it edits values in place.

PREREQUISITE	REQUIREMENT
Docker Desktop	WSL2 backend, at least 8 GB memory — every agent run provisions a container computer.
Node.js	22.22+, 24+ or 26+; pnpm 9+.
Model access	OpenRouter key in <code>workspace/.env</code> , or a local model server.
Supabase	Project URL, anon key and service role key in <code>.env</code> and <code>workspace/.env</code> .
Network	Phone and PC on the same network — a phone hotspot is more reliable than venue Wi-Fi, which often isolates clients.

16.3 Configuration

```
# workspace/.env - AI engine
PI_DEFAULT_PROVIDER=openrouter
PI_DEFAULT_MODEL=deepseek/deepseek-v4-flash
OPENROUTER_API_KEY=sk-or-...

# identity bridge & competency loop
SIGNUPS_ENABLED=false
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...

# agent computer limits - must not be blank
SAMIKSHA_COMPUTER_MEMORY=2g
SAMIKSHA_COMPUTER_CPUS=2
SAMIKSHA_COMPUTER_PIDS_LIMIT=512

# on-premises / offline inference instead
PI_DEFAULT_PROVIDER=local
SAMIKSHA_LOCAL_MODELS=qwen3:1.7b
```

16.4 Backend setup

1. Run `supabase/schema.sql` (all migrations in order) in the SQL editor, or apply the migrations with the CLI.
2. Create a private storage bucket named `materials`.
3. Store the model key as an Edge Function secret `OPENROUTER_API_KEY` or in Vault as `openrouter_api_key`.
4. Deploy the Edge Functions.
5. Create the first administrator; all further accounts are provisioned from the app.

16.5 Release pipeline

Pushing a `v*` tag (or a manual dispatch) builds, in parallel and at no cost on public-repository runners: four Android variants (typecheck → prebuild → generated release keystore → Gradle release build → backend verification → rename → attach to release) and the Windows desktop installer (install → Prisma client → typecheck → Electron build → attach). Variants do not fail fast, so one broken build still yields the others. If repository secrets are absent the build falls back to the project's public URL and anon key — an earlier release without them shipped apps that could not reach the backend, and the pipeline now checks for this explicitly.

16.6 Troubleshooting — failures worth knowing in advance

SYMPTOM	CAUSE AND FIX
Local model plainly running but agents get no reply	A container cannot reach the host's <code>127.0.0.1</code> ; use <code>host.docker.internal</code> . The start script handles it.
Sandbox supervisor does not boot	<code>SAMIKSHA_COMPUTER_MEMORY</code> is blank — blank means invalid, not unlimited. The script sets <code>2g</code> .

SYMPTOM	CAUSE AND FIX
Phone loads the page, then every sign-in fails with CORS	BETTER_AUTH_URL / WEB_ORIGIN left on loopback; they must use the LAN address. The script keeps them aligned.
Container fails with "bad interpreter"	CRLF line endings in the computer start script; .gitattributes pins LF.
ENAMETOOLONG during install on Windows	Path length limit; clone to C:\dev\sih2026 and enable long paths.
Dialog does nothing on web	Native alerts are no-ops on web; use confirmAsync / notify .
Hard error on a web screen with embedded content	WebView is native-only; web uses an iframe branch.

Quality & verification

CHECK	HOW	STATUS
Mobile type safety	<code>pnpm typecheck</code> (strict TypeScript); also gated in the release pipeline	Clean
Workspace checks	<code>pnpm check</code> across the monorepo (Turborepo: typecheck, lint, unit tests)	20 passing
FSRS-6 correctness	Interval, stability and difficulty compared with the reference implementation across sequences	Identical
IRT parity	TypeScript and SQL implementations share constants and formulas	Identical numbers
Identity bridge	Valid credentials land in a working workspace with three agents; wrong password and unknown accounts refused; sign-up refused	Verified
Competency loop	A live officer's twelve real gaps rendered into the agent system prompt with post and required levels; delegations recorded and surfaced	Verified on live data
Adaptive assessment, FSRS review, quiz flow with page citations, both dashboards	Exercised end to end against the live backend and in demo mode	Verified
Samiksha AI & AI practice	Live calls from the web build and APK (screenshots in this document are from live responses)	Verified
Release artefacts	Pipeline asserts backend configuration is compiled into each APK	Enforced
Security advisories	Function grants hardened; RLS on every table; secrets scan in env sync	Applied

Issues found and fixed — and what they taught

ISSUE	RESOLUTION
Every signed-in table read failed with "permission denied for function is_staff"	Hardening had revoked EXECUTE on RLS helper functions from signed-in users; demo mode hid it. Migration 0015 restored the grants.
Released APKs could not reach the backend	Repository secrets were absent at build; the pipeline now falls back to the public URL and anon key and fails loudly if still missing.
Long mock tests stalled past the 150-second function limit	Split into three 5-question batches generated in parallel and hedged against stalls.

ISSUE	RESOLUTION
Video platform blocks cloud requests	The phone fetches the video description and sends it as source text; the browser falls back to the title and the screen says "On the topic".
Dialogs and embedded video broke on web	Cross-platform dialog helpers; iframe branch for web.
Android release build issues in CI	Signing properties read correctly, gradle.properties newline fix, raised timeout, Gradle caching.

What has been done, and why

The platform was built in focused increments. Each milestone below records what was delivered and the reason behind it.

MILESTONE	WHAT WAS DONE	WHY
1. Data foundation	Schema for identity, FRAC competency, content, assessment, learning engine, analytics and AI audit; RLS on every table; first AI functions.	Security and the competency model had to be right before any screen — every later feature reads from them.
2. Mobile app & design system	Expo app, monochrome tokens, UI kit, SVG charts, floating mascot.	A calm, legible interface for daily use by officers in the field; the mascot makes help contextual rather than hidden.
3. Prerequisites & misconceptions	Competency DAG with readiness; distractor-level misconception events.	"Sort by biggest gap" recommends unlearnable things; a score cannot say what someone believes instead.
4. Closing gaps against SIH26101	Technical and digital-governance competencies; PPTX/DOCX extraction; iGOT adapter on the real contract; learning-science constraints.	A line-by-line reading of the statement showed the modern skill families and slide-deck uploads were required, not optional.
5. Single-model gateway & hardening	One pinned open-weight model; schema deployed live; function grants hardened.	Predictable cost and behaviour, a fine-tuning and sovereignty path, and a smaller attack surface.
6. Motion system	Physics-based animation across the app.	Feedback that feels responsive on low-end Android devices without blocking the UI thread.
7. Administrator app, adaptive test, presence, provisioning	Separate admin application; IRT diagnostic; heartbeat presence with realtime; admin-only account creation.	The Ministry's questions are aggregate; measurement must be efficient enough that officers finish it; government systems cannot allow self-registration.
8. Web & demo robustness	Cross-platform dialogs, demo assets on web and APK.	Evaluation happens on whatever device is at hand; the demo must not depend on venue Wi-Fi.
9. AI Workspace on Samiksha identity	Desktop agent workspace, identity bridge, three statistical agents, no onboarding.	Capacity building should also raise productivity — and delegation is itself evidence about competency.
10. The competency loop	Gaps in agent prompts; delegation records; dependency view; "What you are handing to your agents"; mascot opens the Workspace.	Joining learning and doing is what separates this from a quiz generator with a chatbot attached.
11. Portability & one-command start	Windows runbooks, LF pinning, local inference, LAN-aware start script.	The demonstration machine is Windows with Docker; four files must agree on one network address.

MILESTONE	WHAT WAS DONE	WHY
12. Release pipeline	Four signed APK variants and a Windows installer per release.	Evaluators install and run the product directly, with no build environment.
13. Department material & Samiksha AI	Publishing to departments/officers; assistant with learner tutor and operator tools; demo APKs.	Training cells distribute handbooks and circulars; administrators need answers and actions without navigating eight screens.
14. AI practice & one-screen onboarding	Quiz, flashcards and mock test from any material; onboarding reduced to role and goal.	Every assigned document should become practice; provisioning already knows the rest, so first use takes two taps.
15. Release connectivity fix	Build-time backend fallback and verification.	An app that cannot reach its backend is worse than no app at a demonstration.

Limitations & roadmap

Stated plainly, because a platform for official statistics should be as honest about its own uncertainty as it asks officers to be.

LIMITATION	DETAIL	NEXT STEP
iGOT runs on a simulator	The adapter targets the real API contract and eligibility rules, but production credentials need an institutional arrangement.	Obtain credentials; set <code>IGOT_MODE=live</code> .
FRAC mapping is a reconstruction	Grounded in published MoSPI capacity-development material, NSSTA curricula, SQAF and NMDS 2.0; the authoritative FRAC dictionary is internal to DoPT.	Import the official FRAC dictionary; mappings are data, not code.
FSRS weights are population defaults	Per-learner optimisation needs several hundred reviews.	<code>review_logs</code> already captures what optimisation needs.
IRT discrimination held constant	Per-item estimation would overfit on early data.	Estimate per-item parameters once response volume allows.
Base model not yet fine-tuned	Prompts, schemas and grounding checks deliver quality today.	Fine-tune on collected, trainer-approved data (§9.2) and host on government infrastructure.
Agent computers need Docker on the host	Browser, terminal, desktop and recording run in containers; the Workspace UI loads without Docker but agents cannot act.	Managed container hosting for department-wide rollout.
Video grounding	Caption tracks are not available to servers; practice from YouTube uses the description, and discloses it.	Transcripts supplied by trainers for key videos.
Interface language	Generation supports a preferred language; the interface is English today.	Hindi and regional-language interface.

Roadmap

Pilot

- One state DES pilot
- Live iGOT credentials
- Official FRAC import
- Trainer review workflow for generated questions

Scale

- Fine-tuned model on government infrastructure
- Hindi and regional languages
- Personalised FSRS weights
- Offline sync for field staff

Ecosystem

- Karmayogi competency passbook sync
- Training-impact evaluation for commissioned programmes
- Department agent templates (CPI, ASI, PLFS)

Anticipated questions

Q How is this different from a quiz generator with a chatbot?

Three ways. Gaps are computed deterministically against the officer's post, with measurement confidence. Recommendations respect a prerequisite graph and learning science. And the AI Workspace feeds delegation back as competency evidence — so the platform knows not just what an officer scored, but which gaps are costing them autonomy at work.

Q Is the competency gap just the AI's opinion?

No. It is a SQL function: required level minus evidence-weighted score, multiplied by importance and criticality, damped by confidence. The AI only explains that result. Two officers with identical records get identical gaps, which makes every assignment defensible.

Q How do you stop the AI from generating wrong or hallucinated questions?

Strict JSON schemas guarantee structure; each question must include a verbatim quote from the source that is mechanically searched for in the document and resolved to a page or slide; options are validated; failures are flagged and demoted, and the grounding rate is reported. Live difficulty is recalibrated from real responses.

Q Why DeepSeek V4 Flash and not a bigger closed model?

It is open-weight, so it can be fine-tuned on MoSPI/NSSTA material and hosted on government infrastructure for data sovereignty; it is efficient and inexpensive enough for per-officer generation across the whole workforce; its very long context fits whole handbooks; and it supports strict structured output and tool calling. The gateway keeps it swappable.

Q Does iGOT integration actually work?

The adapter is built on Karmayogi's documented production endpoints, including the eligibility rules and the category-filter pitfall. Without institutional credentials it runs a simulator catalogue built from NSSTA/MoSPI curricula; one environment variable switches it to live.

Q What about data privacy and the DPDP Act?

Row Level Security on every table, no secrets in the app, no self-registration, aggregate-only administrator views, private storage, audited AI calls, demo isolation, and a path to host the model within Indian government infrastructure.

Q Can administrators see what an individual officer is doing?

Within their organisation, administrators see a roster and officer record for managing training (presence, gaps, misconceptions, next step). Workforce Intelligence is aggregate by construction and cannot identify individuals. Officers cannot see each other.

Q How does it scale, and what does it cost?

Postgres with indexed, set-based SQL for scoring; serverless functions for AI; extraction cached per material so the second officer pays nothing; a quiz from a full PDF costs well under a cent in model usage. Adaptive testing halves assessment time per officer.

Q What if connectivity is poor?

The scheduler and IRT maths run on the device; demo builds work fully offline; the start script supports a phone hotspot; and local inference allows an on-premises deployment without internet.

Q Why can't officers register themselves?

Anyone with the APK could otherwise appear in a ministry's competency statistics. Accounts tied to verified employee IDs make every record correspond to a real post. The same credentials work in the phone app and the desktop Workspace.

Q Can an AI agent change official figures or send a report?

No. Every agent instruction ends at a draft; the Data Quality agent proposes corrections without applying them; the Report Assistant never sends anything. Administrator AI actions require explicit confirmation.

Q How do you know the adaptive test is accurate?

It uses the standard 2PL IRT model with maximum-information item selection and a precision-based stopping rule, showing its uncertainty band rather than a falsely precise score. The same maths runs in the app and in the database.

Q Why FSRS instead of simpler spaced repetition?

FSRS models stability, difficulty and retrievability explicitly and needs roughly 20–30% fewer reviews than SM-2 for the same retention. Our implementation matches the reference intervals exactly.

Q What happens with scanned PDFs?

If pages yield implausibly little text, they are escalated to vision OCR when a vision-capable model is configured; otherwise the upload is rejected with an actionable message instead of silently generating from an empty document.

Q How does an administrator know training is working?

Quarter-on-quarter competency change per competency and office, officials below the bar, material open rates, and — per officer — confidence rising as evidence accumulates and recurring misconceptions disappearing.

Q Which content types can be uploaded?

PDF, PowerPoint (slides and speaker notes), Word, plain text, images (with OCR), video files up to 50 MB, YouTube links and web links.

Q Is the mobile app a wrapper around a website?

No — it is a native React Native application with its own screens, animation and device capabilities; the same code also produces a web build.

Q What is running today versus planned?

Running: identity bridge, competency loop in both directions, agent provisioning, adaptive assessment, FSRS review, grounded quizzes with citations, AI practice from material, Samiksha AI for officers and administrators, both dashboards, publishing, presence, and the release pipeline. Planned: live iGOT credentials, official FRAC import, model fine-tuning, languages (Section 19).

Glossary

MoSPI	Ministry of Statistics & Programme Implementation.
NSO	National Statistics Office.
NSSTA	National Statistical Systems Training Academy; TPAC — its training programme calendar.
DES	State Directorate of Economics & Statistics.
iGOT Karmayogi	Integrated Government Online Training platform under Mission Karmayogi.
FRAC	Framework of Roles, Activities and Competencies.
SQAF	Statistical Quality Assurance Framework.
NMDS 2.0	National Metadata Structure.
PLFS · NSS · ASI · IIP · CPI · WPI	Periodic Labour Force Survey · National Sample Survey · Annual Survey of Industries · Index of Industrial Production · Consumer / Wholesale Price Index.
DPDP Act	Digital Personal Data Protection Act, 2023.
RLS	Row Level Security — database policies that decide which rows each user may read or write.
Edge Function	Server-side function running close to the database, holding secrets clients never see.
RAG	Retrieval-augmented generation — answering from retrieved passages of source material.
IRT (2PL)	Item Response Theory with difficulty and discrimination parameters.
FSRS	Free Spaced Repetition Scheduler, a memory model for scheduling reviews.
ZPD	Zone of proximal development — what is learnable next given current ability.
Bloom's taxonomy	Levels of cognitive demand: remember, understand, apply, analyse, evaluate, create.
LoRA / QLoRA	Parameter-efficient fine-tuning methods that train small adapter weights on top of a base model.
Open-weight model	An AI model whose trained weights are published, allowing fine-tuning and self-hosting.
MCAR / MAR / MNAR	Missing completely at random / at random / not at random.
PPS · SRS	Probability proportional to size · simple random sampling.

Samiksha · समीक्षा

Built for Smart India Hackathon 2026, Problem Statement SIH26101, Ministry of Statistics & Programme Implementation. The measure of success is not how much work the AI does, but how much more capable each officer becomes.